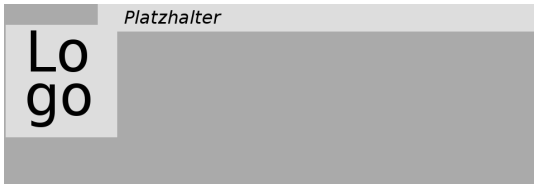


Term paper

Sample title for a report

Author:	Keno Gerken
Matriculation No.:	xxxxxxx
Course of Studies:	Master Maschinenbau
First examiner:	Prof. Dr. Elmar Wings
Submission date:	May 5, 2026

University of Applied Sciences Emden/Leer · Faculty of Technology · Mechanical Engineering Department
Constantiaplatz 4 · 26723 Emden · http://www.hs-emden-leer.de

Contents

List of Figures

List of Listings

Acronyms

CNC Computerized Numerical Control

SPS Speicherprogrammierbare Steuerung

Part I.

Arduino IDE

1. Arduino IDE 2.3.x

This chapter provides a comprehensive examination of the Arduino IDE, covering its features, interface options, and functionality. It includes a detailed explanation of each component's purpose and usability, a step-by-step installation guide, and instructions for initializing and configuring the environment. This foundational overview equips readers to effectively use the IDE for programming and troubleshooting Arduino-compatible hardware.

1.1. Arduino IDE Description

The Arduino IDE is an open-source official software designed for editing, compiling, and uploading code to Arduino modules. It is cross-platform and compatible with operating systems such as Windows, Linux, and macOS. Built on the Java platform, the IDE supports various Arduino modules and programming in C and C++.
The microcontrollers on Arduino boards are programmed to process information provided in the form of code. Programs written in the IDE are referred to as sketches. These sketches generate a Hex file, which is then transferred and uploaded to the microcontroller.
The IDE environment consists of two primary components: the editor and the compiler. The editor is used to write code, while the compiler compiles and uploads the code to the Arduino module.[FAD18] In version 2.3.3 of the Arduino IDE, the menu bar, located at the top of the interface, plays a crucial role by providing access to important commands such as Upload, Verify/Compile, and Save. Additionally, it includes the File menu for opening new or existing files and the Examples section, which offers prewritten sketches for various applications like Blink and Fade.

The 6 buttons are present on top of the screen are as follows:

Figure 1.1.: Menu Button

- The icon check mark is used to Verify the written code. After the code is entered, selecting this icon checks for any errors and confirms that the code is properly structured. This step ensures the code is ready for use with the hardware.
- The icon Upload in the Arduino IDE compiles your code and uploads it to the connected board, making it ready to run.
- The icon Start Debugging icon in the Arduino IDE initiates a debugging session, enabling you to analyze your code by setting breakpoints, stepping through execution, and inspecting variables on compatible boards.
- The icon Serial Plotter in the Arduino IDE opens a tool that visualizes real-time data sent from the board over the serial connection, displaying it as graphs for easier analysis.

The icon Serial Monitor in the Arduino IDE opens a terminal to view and send text-based data over the serial connection, enabling communication with the connected board in real time.

WS:hier noch einmal die letzten Grafiken in der Auflistung nach links verschieben

1.2. Installation of the Arduino IDE

1.2.1. Download of the Arduino IDE

The Arduino Nano 33 BLE Sense utilizes the Arduino Software **Integrated Development Environment (IDE)** for programming, which is the most widely used IDE for all Arduino boards and can be run both online and offline. This open-source Arduino IDE simplifies the process of writing code and uploading it to the board. Various versions of the software are available for each operating system (OS), including macOS, Linux, and Windows. The Arduino community also offers an online platform for coding and saving sketches in the cloud. This online Arduino editor is the latest version of the IDE, featuring all libraries and support for new Arduino boards. To access these software packages, visit the following Website: Arduino-Software, to stay up to date, as there are daily updates available on the mentioned link.

The editor can be downloaded directly from the Arduino Software page with ease. The download options can be seen in Figure 1.2.

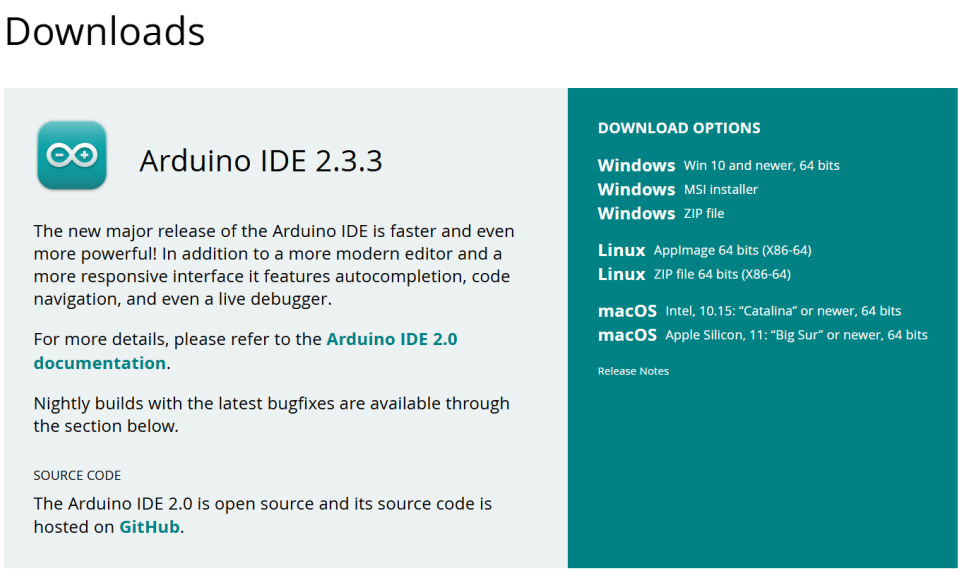


Figure 1.2.: Arduino IDE 2.3.3 Download

1.2.2. Summary of Options

Below is a summary 1.3 of the available download options for the Arduino IDE, tailored to different operating systems and individual needs. Choose the appropriate version for the device to ensure compatibility and access to the latest features.

Summary of Options:

Operating System	Option	Description
Windows	Windows 10 and newer (64-Bit)	Direct download for Windows 10 and newer 64-bit versions, without using the MSI installer.
	MSI Installer	Easy installation through an `.msi` package.
	ZIP File	Portable, no installation required, just extract and run directly.
Linux	AppImage 64-Bit (x86-64)	Portable, no installation required.
	ZIP File 64-Bit (x86-64)	Portable, extract and run directly.
macOS	macOS Intel (10.15: Catalina or newer)	For Intel Macs with macOS 10.15 or newer.
	macOS Apple Silicon (11: Big Sur or newer)	For Apple Silicon Macs (M1, M2) with macOS Big Sur 11 or newer.

Figure 1.3.: Summary of Download Options

1.2.3. Installation on Windows

The installation process for Arduino IDE 2 on a Windows computer will be described step by step in the upcoming section. Following the installation instructions, the subsequent chapter will provide an overview of the IDE's features, including the Board Manager, Library Manager, Sketchbook, and more.

To install the Arduino IDE 2 on a Windows computer, simply run the file downloaded from the software page Arduino Software Arduino-Software. Follow the installation steps, as shown in the Figure **??**, to ensure correct installation.

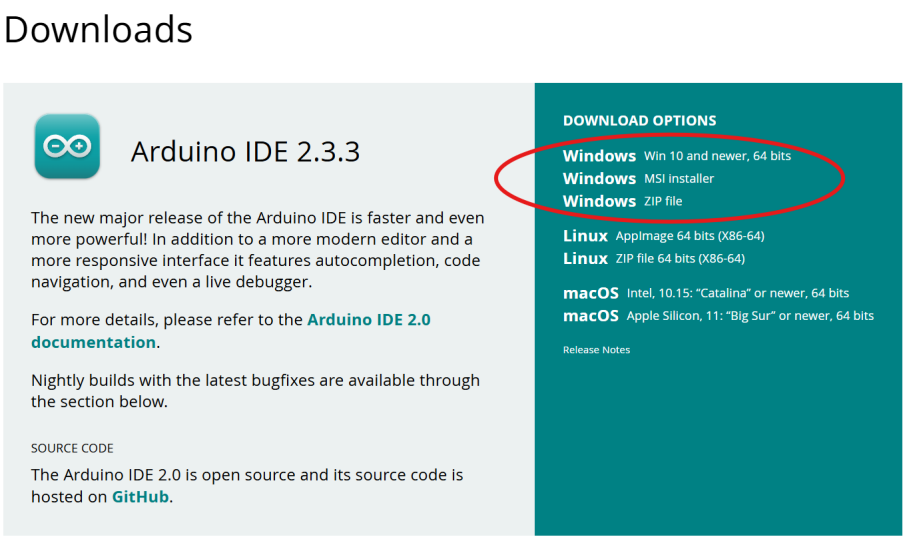


Figure 1.4.: Arduino IDE Create Agent Installation

After the download is complete, open the file setup and proceed with the installation. Select all components as shown in Figure 1.5 in the dialog box, and then click Next .

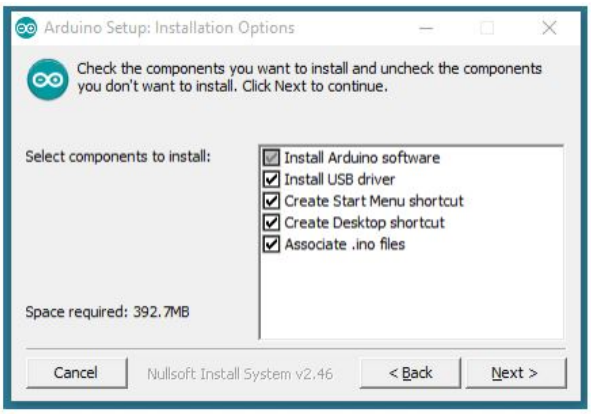


Figure 1.5.: Arduino Setup Installation options

Select the destination folder as seen in Figure 1.6 and click Install.

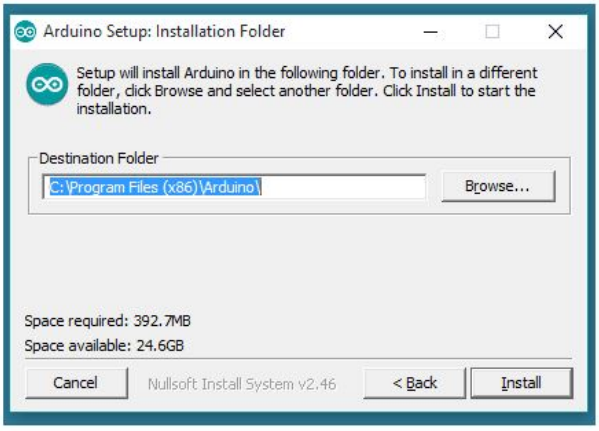


Figure 1.6.: Arduino Setup Installation Folder

Once the installation is complete, open the Arduino IDE. A default sketch will appear on the screen, as shown in Figure 1.7.

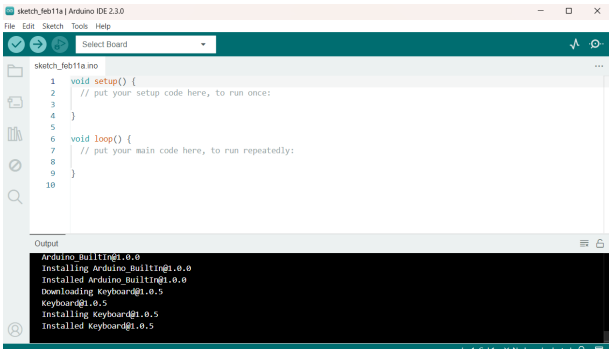


Figure 1.7.: Arduino IDE Sketch

It can be seen from the above figure 1.7 that the basic arduino sketch has two parts. The first part is the function `void setup()` which returns void and we do the intiliaztion such as the output LED color, specifying the core etc. The second part is the function `void loop()` where we define functions which are to be performed through out the

loop. These codes are placed between paranthesis {} and each function has a return type, here it has void return type.
Below in Figure 1.8 is a summary of the installation steps for the Windows version, outlining the key actions required to properly install the softwar

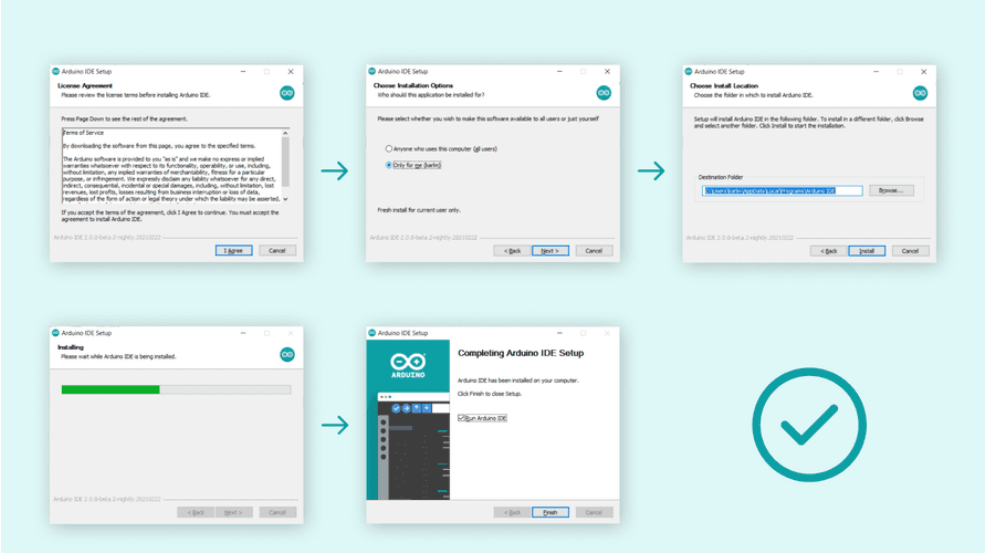


Figure 1.8.: Steps for Installation

2. Arduino IDE 2.3.x - Overview

2.1. Sidebar

The Arduino IDE 2 features a new sidebar, making the most commonly used tools more accessible. 2.1 [Ard24]

- **Verify / Upload** These functions are used to compile and upload code to an Arduino board.
- **Select Board and Port** detected Arduino boards automatically show up here, along with the port number.
- **Sketchbook** This section serves as a centralized repository for all sketches that are stored locally on the user's computer. The Sketchbook is designed to provide a structured, easily accessible space for managing, organizing, and editing code developed within the Arduino environment. Additionally, the Sketchbook offers a synchronization feature with the Arduino Cloud, enabling seamless access to stored sketches across devices. This cloud integration allows users to retrieve and edit sketches from the online Arduino environment
- **Boards Manager** browse through Arduino and third party packages that can be installed. For example, using a MKR WiFi 1010 board requires the Arduino SAMD Boards package installed.
- **Library Manager** browse through thousands of Arduino libraries, made by Arduino and its community.
- **Debugger** test and debug programs in real time.
- **Search** search for keywords in the written code.
- **Open Serial Monitor** opens the Serial Monitor tool, as a new tab in the console.

Figure 2.1.: Overview

2.2. Features

The Arduino IDE 2 is a versatile development tool offering features like direct library installation, cloud synchronization for sketches, and built-in debugging tools. This section highlights some of its core features, with links to more detailed resources.

2.2.1. Sketchbook

The Sketchbook is where Arduino code files, known as sketches, are stored. These files are saved with the extension .ino. To maintain proper organization, each sketch must be placed in a folder named exactly the same as the sketch file. For example, a sketch named `MySketch.ino` must be saved in a folder named `MySketch`. This naming convention is essential for the Arduino IDE to correctly identify and manage the sketches. As seen in Figure 2.2.

Typically, sketches are stored in a folder named `Arduino` located within the Documents directory of the system.

To access the Sketchbook, the icon folder in the sidebar of the Arduino IDE can be selected. This action opens the directory where the sketches are stored, allowing for efficient management and access to the sketches.

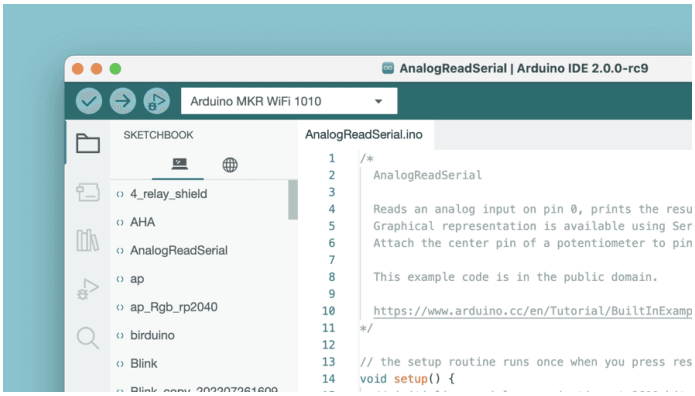


Figure 2.2.: Sketchbook

2.2.2. Boards Manager

The Board Manager can be found in the following steps: Tools > Board > Board Manager.

The Boards Manager in the Arduino IDE allows the installation of different board packages, as shown in Figure 2.3. A board package provides the necessary files and instructions to compile and upload code to specific types of Arduino boards.

Various board packages are available, such as avr, samd and megaavr. Each package supports different Arduino board families. For example, avr is for older boards like the Arduino Uno, while samd is used for newer boards like the Arduino Zero. Using the Boards Manager ensures that the right tools and files are installed for programming the selected board.

Figure 2.3.: Board Manager

2.2.3. Library Manager

The Library Manager can be found in the following steps: Tools » Manage Libraries. The Library Manager in the Arduino IDE allows users to browse and install a wide range of libraries. Libraries extend the core Arduino functions, making it easier to perform tasks such as controlling servo motors, reading data from specific sensors, or working with modules like Wi-Fi. These libraries provide prewritten code to handle specific hardware components and simplify complex tasks, allowing for faster and more efficient development in Arduino projects. As seen in Figure 2.4.

Figure 2.4.: Library Manager

2.3. Integrating and Using a Custom Library in Arduino IDE

In the development of embedded systems using the Arduino platform, libraries are essential for abstracting and simplifying complex functionality. A custom library is particularly beneficial when a specific functionality or hardware interface is repeatedly used across different projects. This section outlines the process of creating, installing, and using a custom library within an Arduino project, in the Arduino Integrated Development Environment (IDE).

2.3.1. Creating the Library Files

A well-structured custom Arduino library typically consists of at least two key components:

Header File (.h): This file contains the declarations of classes, functions, and variables that define the interface of the library. It provides the necessary definitions for the functions and classes to be used by external programs.

Source File (.cpp): This file contains the actual implementation of the functions and methods declared in the header file. It defines the behavior of the functions and classes. The library structure is organized in a way that allows for clear separation between the declaration and implementation of its functionalities.

2.3.2. Installation and Integration of the Library

Once the custom library has been created, it must be properly installed and integrated into the Arduino IDE to be used in projects.

To install a custom library in Arduino follow these steps:

1. Locate the Libraries Folder: The Arduino IDE searches for libraries in the libraries folder, which is located within the Arduino sketchbook directory. The typical path to this directory is:

   ```
   C:/Users/<username>/Documents/Arduino/libraries
   ```

2. Copy the custom library folder, e.g., `Nano33BLESenseLE`, into the directory `libraries`. The final path should look like this:

   ```
   C:/Users/<username>/Documents/Arduino/libraries/Nano33BLESenseLED
   ```

3. Restart the Arduino IDE: After placing the library in the correct directory, restart the Arduino IDE to ensure that it recognizes the newly added library.

2.3.3. Using Symbolic Links for Library Integration

In some cases, it may be more convenient or necessary to store the library files outside the default library directory. For example, if the libraries are stored in a GitHub repository or for better version control, the command mklink can be used to create a symbolic link. This allows the Arduino IDE to access the library files as if they were in the default directory, without duplicating the files.

2.3.4. Windows Tool `mklink` to Create Symbolic Links

Symbolic links allow libraries to be stored in a different location while still being treated by the Arduino IDE as if they reside in the standard directory `libraries`. This can be particularly useful for version-controlled environments, such as when using Git, or to organize libraries without duplicating files.

Steps for Creating a Symbolic Link:

1. **Open Command Prompt with Administrator Privileges:** Press Win + S and search for cmd, click on Command Prompt and select Run as Administrator as seen in Figure 2.5

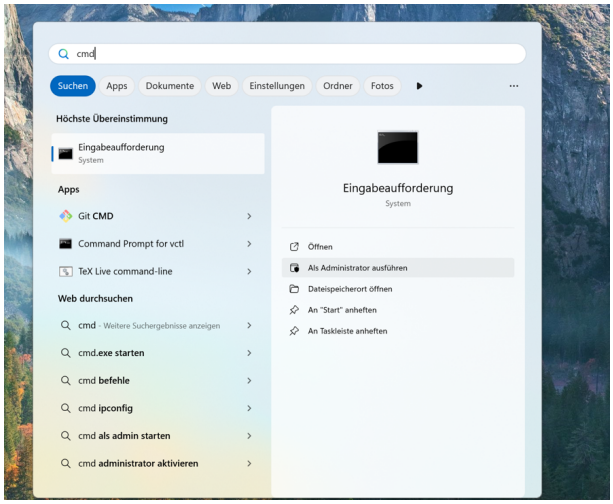


Figure 2.5.: Command Prompt with Administrator Privileges

WS:Figure 1.17, 1.18 wi
an english sytem, so th
figures are on english

2. **Execute the Command mklink :** The command for creating a symbolic link is: `mklink /D "TargetPath" "SourcePath"`
   - **TargetPath:** The location where the Arduino IDE expects the library to be, here `libraries`.
   - **SourcePath:** The actual location of the library.

For this example, the library is located at:
`C:/Users/MSRLabor/Documents/GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/Nano33BLE`

The target path in the Arduino IDE's libraries folder is:
`C:/Users/MSRLabor/Documents/Arduino/libraries/Nano33BLESenseLED`

The full command can be seen in Figure 2.6.

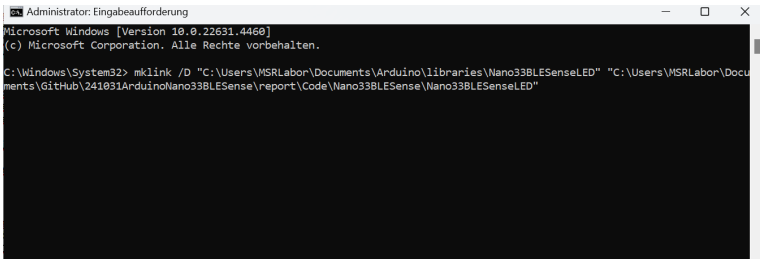


Figure 2.6.: Command Prompt

Verifying the Symbolic Link: After executing the command, navigate to the folder `libraries`. A folder named `Nano33BLESenseLED` should be present, acting as a symbolic link pointing to the actual library location.

2.3.5. Verifying Library Availability in the Arduino IDE

To ensure that the custom library has been successfully integrated and is recognized by the Arduino IDE, follow these steps:

1. Restart the Arduino IDE: If the Arduino IDE was open during the library installation or after creating the symbolic link, close and reopen it. This step ensures the IDE reloads its list of available libraries.

2. Check the Library Menu: Navigate to Sketch go to Include Library and select the Library `Nano33BLESenseLED`, as seen in Figure 2.7

3. Verify Inclusion in the List: If the library is listed, it indicates successful recognition by the IDE. If not, double-check the directory structure and symbolic link to confirm they are correctly configured.

4. Compile a Test Program: Write a simple test program using functions or classes from the library. Compile the sketch to ensure there are no errors, which confirms that the library has been successfully integrated.

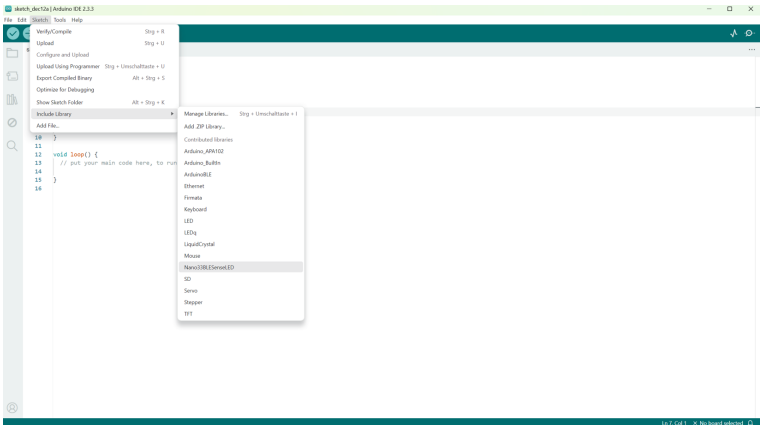


Figure 2.7.: Including the Nano33BLESenseLED Library

Once the program compiles without errors, the library is ready to use, and you can confidently include it in your Arduino projects.

2.3.6. Serial Monitor

The Serial Monitor can be found in the following steps: Tool > Serial Monitor
The Serial Monitor is a tool that allows to view data streaming from the board, via for example the command `Serial.print()`.
Historically, this tool was located in a separate window but is now integrated with the editor, making it easy to run multiple instances simultaneously on your computer. The Serial Monitor can be seen in Figure 2.8.

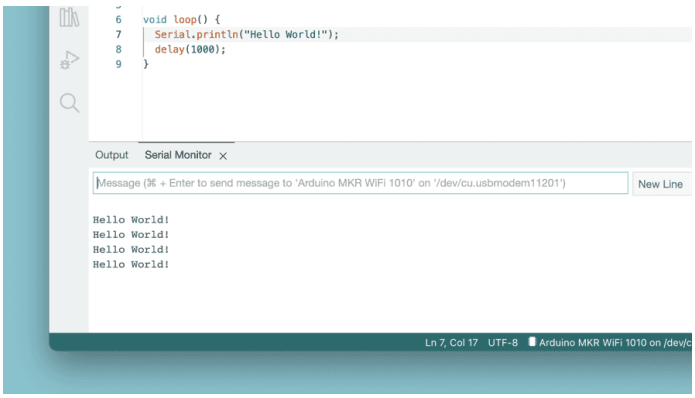


Figure 2.8.: Serial Monitor

2.3.7. Serial Plotter

The tool Serial Plotter is great for visualizing data with graphs and monitoring, such as voltage peaks. It can monitor multiple variables simultaneously, with the option to enable specific types.

2.4. Examples

A significant component of the Arduino Documentation is the set of example sketches bundled with various libraries. These examples demonstrate the practical application of library functions, showcasing their intended uses and main features. Libraries included with certain board packages may also contain their own example sketches. To access these example sketches whether from libraries installed manually or those included with board packages navigate to File » Examples, and locate the desired library from the list.
For instance, when an UNO R4 WiFi board is connected, the examples list appears with options specific to that board. An example sketch demonstrating a pre-loaded Tetris animation can be accessed by navigating to File » Examples » LED Matrix » MatrixIntro and uploading it to the connected board. This example shows how the board's bundled code can be used in practice, assisting users in learning how to control various hardware functions directly.

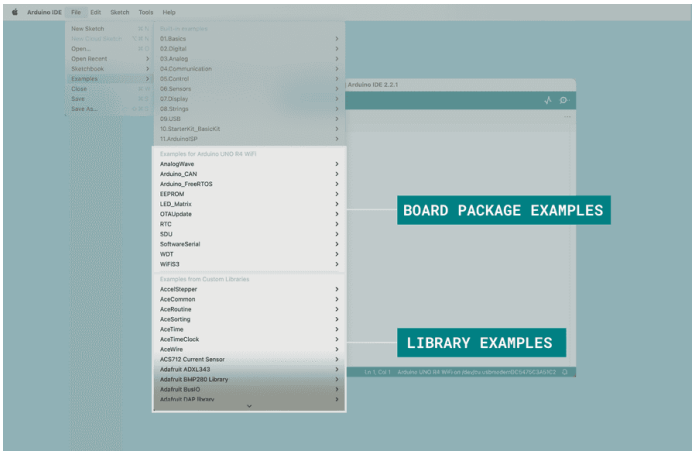


Figure 2.9.: Examples

In Figure 2.9 above, the examples list is shown when a UNO R4 WiFi board is connected to the computer.

2.5. Debugging

The tool Debugger in Arduino IDE 2.3.3 is designed to assist in testing and debugging programs by providing the ability to step through code execution in a controlled manner. This tool allows users to set breakpoints, step through code line-by-line, and inspect variables, helping to identify issues such as logic errors or incorrect variable values. The debugger enables users to pause execution at specific points in the program, allowing for a detailed examination of the program's behavior and memory state. This feature enhances the development process by making it easier to locate and fix errors during runtime, rather than relying solely on print statements or trial and error. 2.10

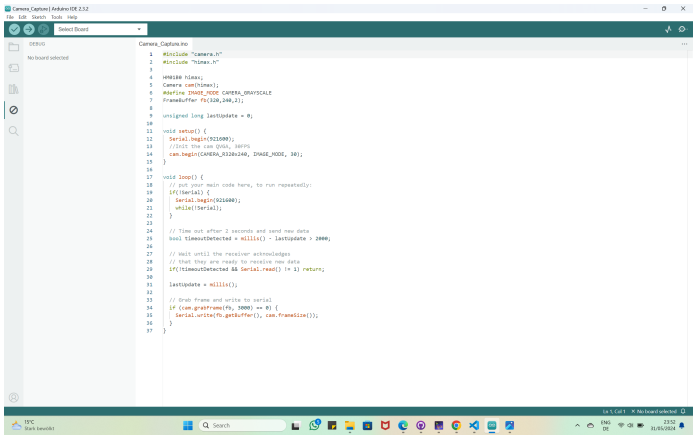


Figure 2.10.: Debugging Example

S:Debugging Bild muss
setzt werden, weil nicht
zu lesen

2.6. Autocompletion

Autocompletion is a key feature in Arduino IDE version 2, making it easier to code by suggesting functions and elements from the Arduino API. This feature helps identify and complete code more quickly, improving efficiency and accuracy.

For autocompletion to work, the board must be selected in the IDE. This ensures that the editor recognizes the correct functions and libraries for the specific board, allowing accurate suggestions. 2.11

Figure 2.11.: Autocomplete

2.7. Conclusion

This guide provides an overview of key features in Arduino IDE 2, along with references to detailed articles for further exploration. Each section aims to enhance understanding and enjoyment of the wide range of functionalities included in the IDE.

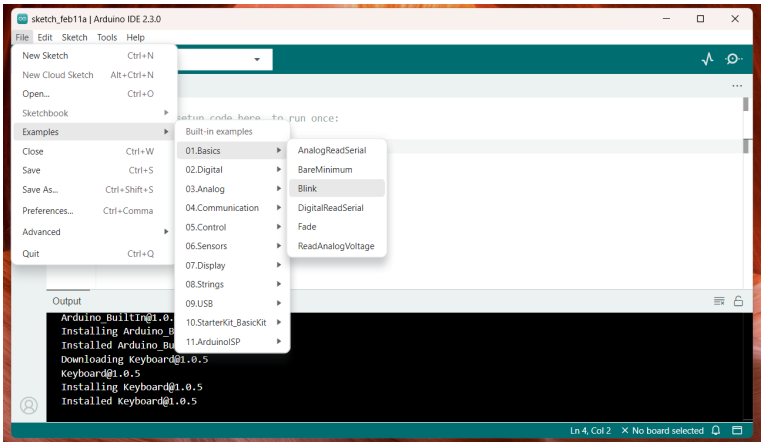


Figure 2.12.: Menu Bar Option

3. Arduino Cloud AI Sketch Generator

Der Arduino Cloud AI Sketch Generator (offiziell meist als Arduino AI Assistant bezeichnet) ist ein Tool innerhalb des Arduino Cloud Editors, das generative künstliche Intelligenz nutzt, um den Entwicklungsprozess zu beschleunigen.

3.1. Der Arduino AI Assistant

Der Arduino AI Assistant stellt eine tiefgreifende Erweiterung der Arduino Cloud dar und basiert technologisch auf dem Sprachmodell *Claude* von Anthropic, welches über *AWS Bedrock* bereitgestellt wird. Ein wesentliches Merkmal dieses Assistenten ist sein spezifisches Training auf Basis von offiziellen Arduino-Dokumentationen, Tutorials und umfangreichen Bibliotheksdaten, wodurch er direkt in den Online-Editor integriert ist und kontextbezogene Unterstützung bietet.
Die Funktionalität des Assistenten umfasst primär die automatisierte Code-Generierung, bei der Nutzer ihre Anforderungen in natürlicher Sprache formulieren können. So ist es beispielsweise möglich, durch eine einfache Beschreibung wie "Erstelle einen Sketch für eine WordClock mit WS2812B LEDs und einem DS3231 RTC Modul" einen vollständigen, funktionsfähigen C++ Code zu generieren. Darüber hinaus unterstützt die KI den Entwicklungsprozess durch intelligentes Debugging, indem sie Fehlermeldungen analysiert und gezielte Korrekturvorschläge unterbreitet.
Ein weiterer Vorteil ist die Fähigkeit zur Code-Erklärung, die es ermöglicht, komplexe Abschnitte zu markieren und deren Wirkungsweise detailliert erläutern zu lassen. Durch das integrierte Kontextbewusstsein erkennt die KI zudem automatisch das gewählte Board sowie installierte Bibliotheken und passt die Code-Vorschläge präzise an die verwendete Hardware-Umgebung an.

3.2. Installation des Arduino AI Assistant

Part II.

Domain Knowledge - Hardware

4. Hypatia Board

WS:Überschriften nochmal anpassen

The Hypatia Board is based on the ESP32-S3-WROOM-1 microcontroller from Espressif. It offers Wi-Fi and Bluetooth Low Energy (BLE) connectivity and can be programmed via USB. In addition to the microcontroller, the board includes the LSM100A module, which supports both LoRa and Sigfox communication standards. The Dimensions and the location of the four mounting holes are shown in the next figure4.1.

WS:Cite Hypatia Handbook

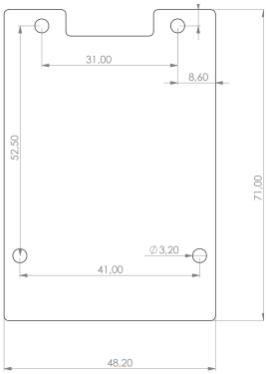


Figure 4.1.: Dimensions Hypatia Board [**esp32s3wroom1:2024**]

4.1. Interface

The Hypatia board's interface includes 34 female header pins for flexible peripheral connections, three JST-PH connectors for power and data interfaces, and two push buttons dedicated to reset and boot functions. Additionally, a WS2812 RGB LED provides operational feedback, enabling monitoring of the board's status.

4.1.1. Pinout

The board features a total of 34 female header pins. Among these, three are connected to GND (Pins 1, 16, and 17 on J1), two provide 3.3V power (Pins 2 and 15 on J1), and one supplies 5V (Pin 17 on J2). Additionally, Pins 3 and 4 on J2 are designated for serial communication via TXD and RXD. The remaining pins serve as general-purpose input/output (GPIO). The diagram below 4.2 provides a overview of which pins are safe to use, and which should be avoided or used with caution.

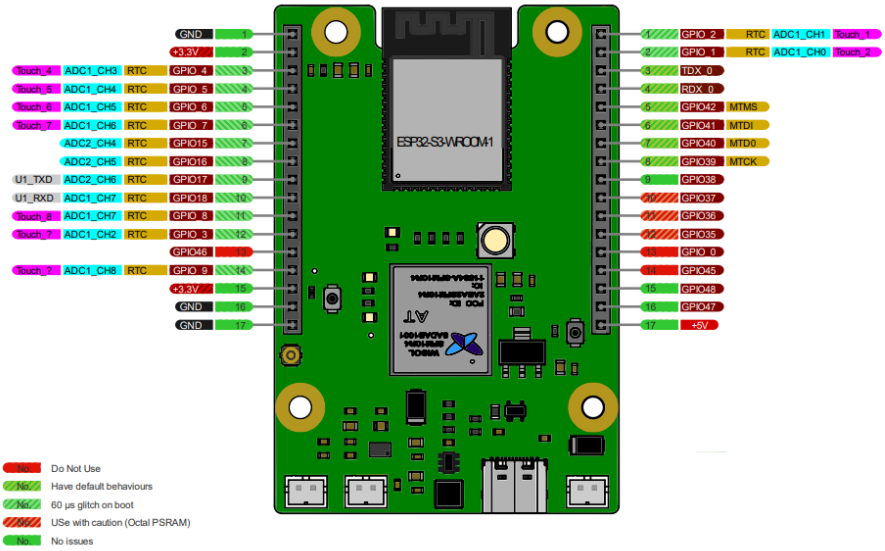


Figure 4.2.: Pinout Hypatia Board

During the power-up phase of the ESP32-S3, certain pins may exhibit glitches, brief voltage disturbances that deviate from their normal, expected state. This happens primarily because the power rails and internal circuits require a short period to stabilize. Until the internal power domains and voltage regulators reach their steady state, the pins connected to these domains may momentarily adopt unintended logic levels. Specifically, some pins are temporarily driven low or high for approximately 60 microseconds as the chip transitions from an unpowered to a powered state. These brief glitches can be seen as a natural consequence of how the chip's internal logic and GPIO matrix handle the initial surge of power. As the system clock and internal logic gates activate and synchronize, these minor fluctuations on the pins are unavoidable. While these glitches are usually short and harmless in typical applications, designers should be aware of them, especially when connecting external components sensitive to these voltage transients. The ESP32-S3's design accounts for these behaviors by specifying which pins experience glitches, enabling developers to plan for any potential unintended effects.
The following two tables provide information on the internal connections of the pins on either J1 or J2 to the ESP32-S3 microcontroller.

4.1.2. JST-PH Connectors

A JST-PH connector is a low-profile wire-to-board connector, designed for secure and space-saving connections on high-density Printed Circuit Boards (PCBs). On the Hypatia Board three JST Connectors with 2 circuits are integrated**??**. This connector family is known for its boxed-shaped shrouded headers[**JST:2024**].

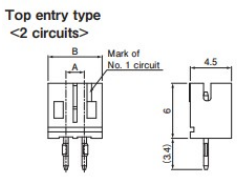


Figure 4.3.: JST-PH Connector Dimension [**JST:2024**]

Table 4.1.: Pinout Hypatia Board J1 [see Syd, p. 7]

Pin J1	Function	Description
1	GND	GND
2	+3.3V	Power supply
3	GPIO 4	RTC_GPIO4,GPIO4,TOUCH4,ADC1_CH3
4	GPIO 5	RTC_GPIO5,GPIO5,TOUCH5,ADC1_CH4
5	GPIO 6	RTC_GPIO6,GPIO6,TOUCH6,ADC1_CH5
6	GPIO 7	RTC_GPIO7,GPIO7,TOUCH7,ADC1_CH6
7	GPIO 15	RTC_GPIO15,GPIO15,U0RTS,ADC2_CH4,XTAL_32K_P
8	GPIO 16	RTC_GPIO16,GPIO16,U0CTS,ADC2_CH5,XTAL_32K_N
9	GPIO 17	RTC_GPIO17,GPIO17,U1TXD,ADC2_CH6
10	GPIO 18	RTC_GPIO18,GPIO18,U1RXD,ADC2_CH7,CLK_OUT3
11	GPIO 8	RTC_GPIO8,GPIO8,TOUCH8,ADC1_CH7,SUBSPICS1
12	GPIO 3	RTC_GPIO3,GPIO3,TOUCH3,ADC1_CH2
13	GPIO 46	GPIO46
14	GPIO 9	RTC_GPIO9,GPIO9,TOUCH9,ADC1_CH8,FSPIHD,SUBSPIHD
15	+3.3V	Power supply
16	GND	GND
17	GND	GND

Table 4.2.: Pinout Hypatia Board J2 [see Syd, p. 8]

Pin J2	Function	Description
1	GPIO 1	RTC_GPIO1,GPIO1, TOUCH1, ADC1_CH0
2	GPIO 2	RTC_GPIO2,GPIO2, TOUCH2, ADC1_CH1
3	TXD0	RTC_GPIO17,GPIO17,U1TXD,ADC2_CH6
4	RXD0	RTC_GPIO18,GPIO18,U1RXD,ADC2_CH7,CLK_OUT3
5	GPIO 42	MTMS,GPIO42
6	GPIO 41	MTDI,GPIO41, CLK_OUT1
7	GPIO 40	MTDO,GPIO40,CLK_OUT2
8	GPIO 39	MTCK,GPIO39,CLK_OUT3, SUBSPICS1
9	GPIO 38	GPIO38,FSPIWP, SUBSPIWP
10	GPIO 37	SPIDQS,GPIO37,FSPIQ, SUBSPIQ
11	GPIO 36	SPIIO7,GPIO36,FSPICLK,SUBSPICLK
12	GPIO 35	SPIIO6,GPIO35,FSPID,SUBSPID
13	GPIO 0	RTC_GPIO0,GPIO0
14	GPIO 45	GPIO45
15	GPIO 48	SPICLK_N_DIFF,GPIO48,SUBSPICLK_N_DIFF
16	GPIO 47	SPICLK_P_DIFF,GPIO47,SUBSPICLK_P_DIFF
17	+5V	

The purpose of these three JST-PH connectors is to integrate a LiPo battery, a solar panel, and an on/off switch, enabling system reboot or reset as required, into the Hypatia circuit **??**.

WS:Add the circuit Diagramm to the switc

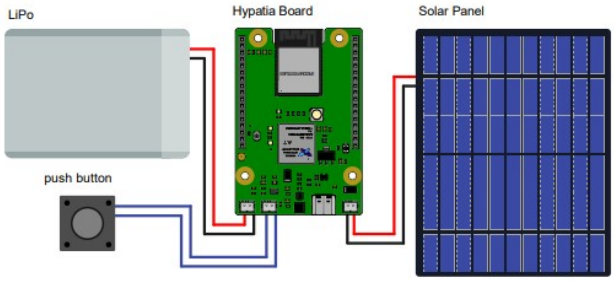


Figure 4.4.: Setup of Hypatia Board with Lipo Battery, Solar panel and On/off Switch [see Syd, p. 2]

4.1.3. Boot Modus

The Hypatia board is equipped with two push buttons: one for reset and one for boot. To upload new code to the board, it must be placed in boot mode. Boot mode is a special state that enables the microcontroller's bootloader to receive new firmware over a serial interface. To set the board into boot mode, press and hold the reset button while briefly pressing the boot button. Once the boot button has been pressed, release both buttons. This sequence ensures that the Hypatia board is ready to accept new code uploads.

WS:Tikz with button location

4.1.4. WS2812 RGB LED

S:Check the Tables and e Text for V_{IH} correct - V_IH wrong

The WS2812B is an intelligent control LED light source, seamlessly integrating a control circuit and RGB chip within a 5050 SMD package 4.5. This compact unit features an intelligent digital data latch, a signal reshaping and amplification driver circuit, an internal oscillator, and a 12V programmable constant-current control system.

S:What is a LED, what does the control circuit, what is a 5050 SMD package. what is that data latch how does the circuit look like, explaination

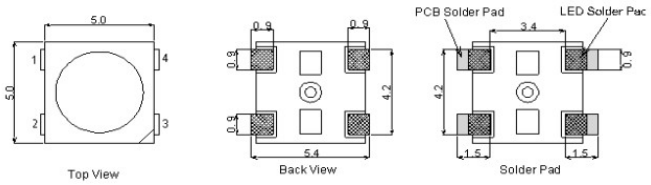


Figure 4.5.: Dimensions of WS2812B RGB LED [**Worldsemi:2013**]

The circuit of the WS2812B includes built-in reverse connection protection, preventing damage if the power supply is accidentally connected in reverse. It also incorporates an electric reset circuit and a power-loss reset circuit for reliable startup and recovery. The control circuit and the LED share a single power source, which simplifies the design and reduces component count. The Pin out and the Pin Functions are presented in the following table 4.3 and visualized in the following picture **??**.

Table 4.3.: PIN function WS2812B [**Worldsemi:2024**]

NO.	Symbol	Function description
1	VDD	Power supply LED
2	DOUT	Control data signal output
3	VSS	Ground
4	DIN	Control data signal input

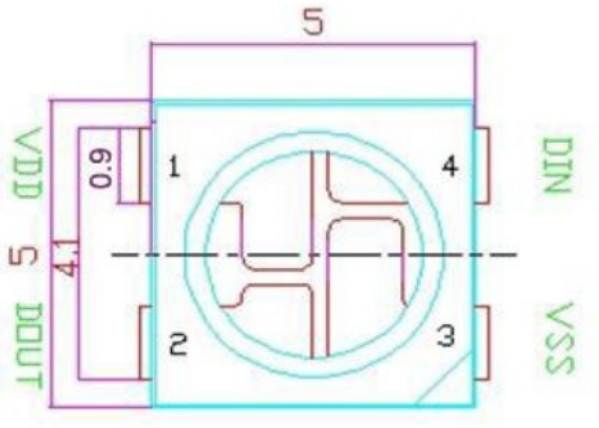


Figure 4.6.: Pinout of WS2812B RGB LED [**Worldsemi:2024**]

The absolute maximum ratings of the WS2812B LED module specify the limits beyond which permanent damage may occur. The power supply voltage, V_{DD}, can range from +3.5V to +5.3V. The input voltage, V_I, can vary from -0.5V up to V_{DD} +0.5V. The module can operate within a junction temperature range of -25 $^{\circ}C$ to +80 $^{\circ}C$, and can be stored safely at temperatures ranging from -40 $^{\circ}C$ to +105 $^{\circ}C$ [**Worldsemi:2024**]. Regarding electrical input parameters, the maximum input current (I_IN) is specified at $\pm 1 \mu A$ under typical conditions ($T_A = -20\,^{\circ}\mathrm{C} \sim +70\,^{\circ}\mathrm{C}$, $V_{DD} = 4.5\,\mathrm{V} \sim 5.5\,\mathrm{V}$, $V_{SS} = 0\,\mathrm{V}$, unless otherwise specified). For the input voltage levels, a logic high (V_{IH}) must be at least 0.7 times the supply voltage (V_{DD}), while a logic low (V_{IL}) must be no more than 0.3 times V_{DD}. The hysteresis voltage (V_H), which defines the difference between switching thresholds to avoid signal noise, has a typical value of 0.35V 4.4.

Table 4.4.: Electrical Characteristics WS2812B [**Worldsemi:2024**]

Prameter	Smybol	conditions	Min	Tpy	Max	Unit
Input current	I_I	$V_I = V_{\mathrm{DD}}/V_{\mathrm{SS}}$			± 1	μA
Input voltage level	V_{IH}	D_IN, SET	$0.7V_{\mathrm{DD}}$			V
	V_{IL}	D_IN, SET			0.3 V_{DD}	V
Hysteresis voltage	V_H	D_IN, SET		0.35		V

Data transmission

The WS2812B supports transmission distances of more than 5 meters between any two points without the need for additional circuitry. At a refresh rate of 30 frames

per second, it supports a cascade length of at least 1024 pixels. Data transmission speeds reach up to 800 Kbps [**Worldsemi:2024**].

To ensure reliable communication between the microcontroller or data source and the WS2812B, regardless of how many LEDs are in the chain, the protocol specifies a bit period of 1.25 μs (with a margin of ±600 ns), and each bit is represented by a distinct high and low voltage timing pattern 4.7.

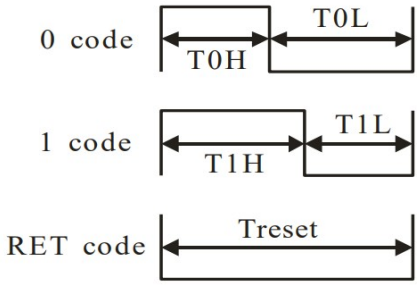


Figure 4.7.: Sequence Times WS2812B [**JST:2024**]

The precise high and low voltage durations for transmitting logical '0' and '1' bits, as well as the reset condition are provided by the following table4.5.

Table 4.5.: Data transfer time for WS2812B (TH+TL=1.25 μs±600 ns) [**Worldsemi:2024**]

Symbol	**Description**	**Typical**	**Tolerance**
TOH	0 code, high voltage time	0.4 μs	±150 ns
T1H	1 code, high voltage time	0.8 μs	±150 ns
T0L	0 code, low voltage time	0.85 μs	±150 ns
T1L	1 code, low voltage time	0.45 μs	±150 ns
RES	Low voltage time	Above 50 μs	

For a '0' bit (shown as "0 code" in the diagram), the data line is held high for 0.4 μs (T0H) and then low for 0.85 μs (T0L). This combination of a short high period followed by a longer low period defines the '0' bit and allows the LED to interpret it correctly.

For a '1' bit (shown as "1 code"), the data line is held high for a longer period of 0.8 μs (T1H), followed by a shorter low period of 0.45 μs (T1L). This pattern of a longer high time and a shorter low time distinguishes the '1' bit from the '0' bit.

The final diagram shows the reset code (RET code). Here, the data line is held low for a minimum of 50 microseconds (Treset). This low voltage period is significantly longer than the timing for individual data bits and signals to all LEDs that a new data transmission cycle will begin. This reset code ensures that all LEDs are synchronized to receive the next series of color data.

Table 4.6.: Switching characteristics of the WS2812B [**Worldsemi:2024**]

Parameter	Symbol	Condition	Min	Typ	Max	Unit
Transmission delay time	t_{PLZ}	CL=15pF, DIN→ DOUT, RL=10kΩ			300	ns
Fall time	t_{THZ}	CL=300pF, OUTR/OUTG/OUTB			120	μs
Input capacity	C_1				15	pF

WS:Make the Caption all the Tables nicer and more accurate

In addition to the carefully defined data transfer timings for each bit ('0' and '1' codes) and the reset condition, the switching characteristics of the WS2812B also play a role in ensuring reliable operation 4.6. These characteristics describe the electrical behavior of the data input and output pins, which directly influences how accurately the LEDs can interpret the incoming data signals. The transmission delay time (tPLZ), which ensures that the data stream propagates quickly from one LED to the next in the chain, amounts to 300 ns. The fall time (tTHZ) determines how quickly the data output can transition between logic levels and is specified as 120 μs for the WS2812B. Finally, the input capacitance (C1) indicates how much electrical load the data input pin of each LED represents, and for this RGB LED, it is 15 pF.

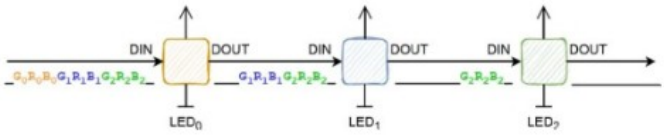


Figure 4.8.: Serial Data transmission between LEDs WS2812B [**Ritschel:2023**]

The diagram 4.8 illustrates the data transmission method of the WS2812B RGB LEDs, which operates based on a single data line. Each LED receives data through the data input pin (DIN), and the data output pin (DOUT) of one LED is connected to the DIN of the next LED in the chain. Data transmission begins with the reset code that marks the start of a new data refresh cycle. In a data refresh cycle, the data for each LED is transmitted sequentially4.9 [**Worldsemi:2013**].

WS:How to do the citation here???! Its discribed in the picture from the datasheet not mentioned in the text from Ritschel

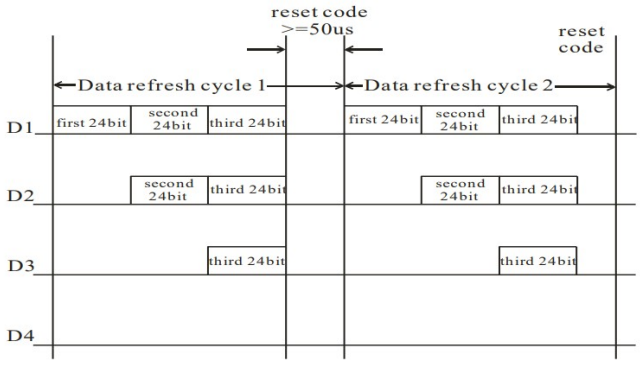


Figure 4.9.: Data transmission method WS2812B [**Worldsemi:2024**]

Each LED requires 24-bit color data, split into 8 bits for green, 8 bits for red, and 8 bits for blue. These data packets are transmitted in succession and determine

the brightness and color of each LED. After a pause, the RGB data is transmitted cyclically to all LEDs. The first LED receives the first 24-bit RGB data and then forwards the remaining data stream to the next LED, which also takes its own 24-bit data and forwards the rest. This approach ensures that each LED receives its unique data 4.9 [**Ritschel:2023**].
Once the data for all LEDs has been transmitted, another reset code (a low signal for at least 50 microseconds) ends the current refresh cycle and prepares the LEDs for the next update. This cycle repeats whenever new color data needs to be sent to the LEDs 4.9.

RGB characteristics

Each pixel, made up of the three primary colors (red, green, and blue 4.7), supports 256 levels of brightness per channel. This allows the WS2812B to achieve a full-color display with 16.777.216 possible colors and a scan frequency of at least 400 Hz per second.

Table 4.7.: RGB IC characteristic parameter WS2812B [**Worldsemi:2024**]

Emitting color	Model	Wavelength	Luminous intensity	Voltage
Red	13CBAUP	620-625 nm	390-420 mcd	2.0-2.2 V
Green	13CGAUP	522-525 nm	660-720 mcd	3.0-3.4 V
Blue	10R1MUX	465-467 nm	180-200 mcd	3.0-3.4 V

Adafruit_NeoPixel Library

WS:Find any discribtion of Library -Git doesnt work

The Adafruit DMA NeoPixel library, version 1.3.3, released on April 24, 2024, and published under the MIT license is specifically designed for NeoPixel DMA support on SAMD21 and SAMD51 microcontrollers using the Arduino platform and is maintained by Adafruit.

WS2812B Example Code for Hypatia

The Hypatia Handbook provides example code to test the WS2812B RGB LED using the Adafruit NeoPixel library 4.1. Utilizing GPIO21 for data transmission within the Arduino environment, the code controls the LED to alternately glow red, green, and blue, changing every 50 ms [see Syd, p. 12].

```
#include <Adafruit_NeoPixel.h>
// Define the pin to which the WS2812B LED is connected
#define LED_PIN 21
// We only have one LED
#define NUM_LEDS 1
// Create a NeoPixel object
Adafruit_NeoPixel strip = Adafruit_NeoPixel(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
void setup() {
// Initialize the LED
strip.begin();
strip.show(); // Make sure all LEDs are off
}
void loop() {
// Set the LED to red
strip.setPixelColor(0, strip.Color(255, 0, 0)); // red
strip.show();
delay(50); // wait for one second
// Set the LED to green
strip.setPixelColor(0, strip.Color(0, 255, 0)); // green
strip.show();
```

```
21 delay(50); // wait for one second
22 // Set the LED to blue
23 strip.setPixelColor(0, strip.Color(0, 0, 255)); // blue
24 strip.show();
25 delay(50); // wait for one second
26 }
```

Listing 4.1: WS2812B Example Code

The code begins by including the Adafruit_NeoPixel Library, which provides functions to control the LED. The LED_PIN is set to 21, specifying the pin on the microcontroller to which the data input of the WS2812B LED is connected. NUM_LEDS is defined as 1, indicating that only a single LED is in use. An NeoPixel Object of the Adafruit_NeoPixel class, named strip, is created with parameters specifying the number of LEDs, the data pin, and the LED color order and protocol speed (NEO_GRB + NEO_KHZ800). This combination tells the library to use green, red, and blue as the order for the color data and to communicate at 800 kHz, matching the WS2812B's protocol. The setup() function runs once when the microcontroller is powered up or reset. Here, strip.begin() initializes the LED data pin and prepares the NeoPixel library for communication. The call to strip.show() ensures that the LED is initially turned off by updating the LED with any pending color changes (which, at this point, are off by default). The loop() function runs continuously after setup() finishes. In this loop, the LED color is changed to red by calling strip.setPixelColor(0, strip.Color(255, 0, 0)), which sets the LED's color to full red with no green or blue. strip.show() is called immediately afterward to send the color data to the LED, making the LED light up red. A short delay(50) is inserted to keep the LED red for 50 ms. The same process is repeated for green and blue colors. strip.setPixelColor(0, strip.Color(0, 255, 0)) changes the LED to green, and after 50 ms, strip.setPixelColor(0, strip.Color(0, 0, 255)) changes it to blue.

4.2. ESP32-S3-WROOM-1

The ESP32-S3-WROOM-1 module is an integrated wireless and microcontroller platform designed for a wide range of embedded applications. It features 2.4 GHz Wi-Fi (802.11b/g/n) and Bluetooth 5 connectivity and is based on the ESP32-S3 series of SoCs, which incorporate a dual-core 32-bit Xtensa LX7 microprocessor. The module supports up to 16 MB of flash and 16 MB of PSRAM, and provides up to 36 GPIOs along with a comprehensive set of digital and analog peripherals and wireless communication is enabled through an on-board PCB antenna.

Figure 4.10.: ESP32-S3-WROOM-1 [see Sys24, p. 1]

4.2.1. Physical Dimensions

Figure 4.11 shows the ESP32-S3-WROOM-1 module with dimensions in millimeters. The module measures 18.0 mm in width and 25.5 mm in length, with a maximum height of 3.1 mm including the shielding. Pin pads are arranged along the sides with a standard pitch of 1.27 mm, while the central pad area on the bottom side measures 7.5 mm ×10.29 mm. The top view includes antenna placement and keep-out zones, while the bottom view indicates pad layout and spacing relevant for PCB footprint design. Tolerances are specified for key dimensions to support accurate mechanical integration.

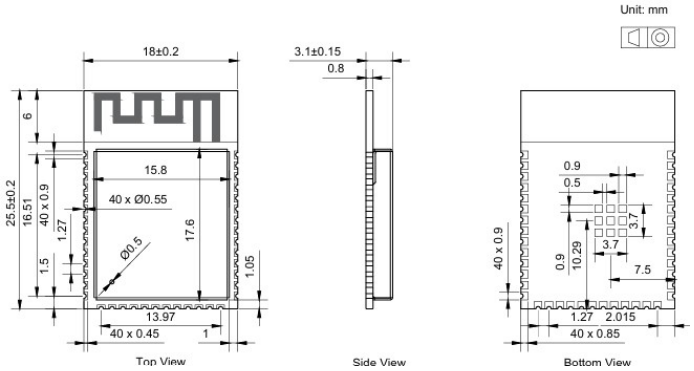


Figure 4.11.: ESP32-S3-WROOM-1 [see Sys24, p. 9]

4.2.2. Block Diagramm

The diagrams illustrate the block structure of the ESP32-S3-WROOM-1 module. Power is supplied via a 3.3 V line. A 40 MHz crystal oscillator provides a stable timing reference for the system. The module incorporates an RF matching circuit connected to an antenna for wireless communications. It also integrates a Quad SPI (QSPI) flash memory interface, which supports optional PSRAM. Several SPI signals, including SPICSO, SPICLK, SPIQ, SPID, SPIWP, and SPIHD, are connected to the QSPI flash interface. The module exposes general-purpose input/output (GPIO) pins for external device interfacing. A dedicated enable (EN) pin allows control of the module's power state.

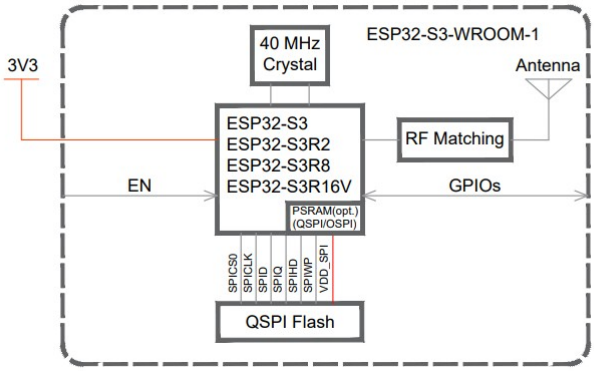


Figure 4.12.: ESP32-S3-WROOM-1 Block Diagramm [see Sys24, p. 9]

4.2.3. Pin Definitions

The module has 41 pins. The Pin Definitions are described in the following Table 4.8.

Table 4.8.: Pin Definitions of the ESP32-S3-WROOM-1 [see Sys24, p. 3]

Name	**No.**	*Type*[a]	**Function**
GND	1	P	GND
EN	3	I	High: on, enables the chip. Low: off, the chip powers off. Note: Do not leave the EN pin floating.
IO4	4	I/O/T	RTC_GPIO4, **GPIO4**, TOUCH4, ADC1_CH3
IO5	5	I/O/T	RTC_GPIO5, **GPIO5**, TOUCH5, ADC1_CH4
IO6	6	I/O/T	RTC_GPIO6, **GPIO6**, TOUCH6, ADC1_CH5
IO7	7	I/O/T	RTC_GPIO7, **GPIO7**, TOUCH7, ADC1_CH6
IO15	8	I/O/T	RTC_GPIO15, **GPIO15**, U0RTS, ADC2_CH4, XTAL_32K_P
IO16	9	I/O/T	RTC_GPIO16, **GPIO16**, U0CTS, ADC2_CH5, XTAL_32K_N
IO17	10	I/O/T	RTC_GPIO17, **GPIO17**, U1TXD, ADC2_CH6
IO18	11	I/O/T	RTC_GPIO18, **GPIO18**, U1RXD, ADC2_CH7, CLK_OUT3
IO8	12	I/O/T	RTC_GPIO8, **GPIO8**, TOUCH8, ADC1_CH7, SUBSPICS1
IO19	13	I/O/T	RTC_GPIO19, GPIO19, U1RTS, ADC2_CH8, CLK_OUT2, **USB_D-**
IO20	14	I/O/T	RTC_GPIO20, GPIO20, U1CTS, ADC2_CH9, CLK_OUT1, **USB_D+**
IO3	15	I/O/T	RTC_GPIO3, **GPIO3**, TOUCH3, ADC1_CH2
IO46	16	I/O/T	**GPIO46**
IO9	17	I/O/T	RTC_GPIO9, **GPIO9**, TOUCH9, ADC1_CH8, FSPIHD, SUBSPIHD
IO10	18	I/O/T	RTC_GPIO10, **GPIO10**, TOUCH10, ADC1_CH9, FSPICS0, FSPIIO4, SUBSPICS0
IO11	19	I/O/T	RTC_GPIO11, **GPIO11**, TOUCH11, ADC2_CH0, FSPID, FSPIIO5, SUBSPID
IO12	20	I/O/T	RTC_GPIO12, **GPIO12**, TOUCH12, ADC2_CH1, FSPICLK, FSPIIO6, SUBSPICLK
IO13	21	I/O/T	RTC_GPIO13, **GPIO13**, TOUCH13, ADC2_CH2, FSPIQ, FSPIIO7, SUBSPIQ
IO14	22	I/O/T	RTC_GPIO14, **GPIO14**, TOUCH14, ADC2_CH3, FSPIWP, FSPIDQS, SUBSPIWP
IO21	23	I/O/T	RTC_GPIO21, **GPIO21**
IO47[c]	24	I/O/T	SPICLK_P_DIFF, **GPIO47**, SUBSPICLK_P_DIFF
IO48[c]	25	I/O/T	SPICLK_N_DIFF, **GPIO48**, SUBSPICLK_N_DIFF
IO45	26	I/O/T	**GPIO45**

Continued on next page

Name	No.	*Type*[a]	Function
IO0	27	I/O/T	RTC_GPIO0, **GPIO0**
IO35[b]	28	I/O/T	SPIIO6, **GPIO35**, FSPID, SUBSPID
IO36[b]	29	I/O/T	SPIIO7, **GPIO36**, FSPICLK, SUBSPICLK
IO37[b]	30	I/O/T	SPIDQS, **GPIO37**, FSPIQ, SUBSPIQ
IO38	31	I/O/T	**GPIO38**, FSPIWP, SUBSPIWP
IO39	32	I/O/T	**MTCK**, GPIO39, CLK_OUT3, SUBSPICS1
IO40	33	I/O/T	**MTDO**, GPIO40, CLK_OUT2
IO41	34	I/O/T	**MTDI**, GPIO41, CLK_OUT1
IO42	35	I/O/T	**MTMS**, GPIO42
RXD0	36	I/O/T	**U0RXD**, GPIO44, CLK_OUT2
TXD0	37	I/O/T	**U0TXD**, GPIO43, CLK_OUT1
IO2	38	I/O/T	RTC_GPIO2, **GPIO2**, TOUCH2, ADC1_CH1
IO1	39	I/O/T	RTC_GPIO1, **GPIO1**, TOUCH1, ADC1_CH0
GND	40	P	GND
EPAD	41	P	GND

The table includes extra pin functions for the ESP32-S3:

- [a]: Pin roles are denoted using the following abbreviations: P for power supply, I for input, O for output, and T for high impedance. Default functions are shown in bold font. For pins 28 to 30, the default function depends on the configuration of the eFuse bit.
- [b]: For modules equipped with Octal SPI PSRAM (such as the ESP32-S3R8 or ESP32-S3R16V), pins IO35, IO36, and IO37 are internally connected to the memory and cannot be repurposed.
- [c]: Additionally, on ESP32-S3R16V modules, GPIO47 and GPIO48 operate at 1.8 V, as determined by the VDD_SPI voltage of the chip. This voltage level differs from the standard operating voltage of other GPIOs on the device.

4.2.4. Boot Configurations

At startup or following a hardware reset, the ESP32-S3 chip automatically configures several boot parameters using a combination of strapping pins and eFuse bits, without requiring any input from the microcontroller. The boot mode is determined by the state of GPIO0 and GPIO46, while the VDD_SPI voltage level is set via GPIO45 or by programming the eFuse bits EFUSE_VDD_SPI_FORCE and EFUSE_VDD_SPI_TIEH. Similarly, control over ROM message printing is handled through GPIO46 and can also be managed using the eFuse settings EFUSE_UART_PRINT_CONTROL and EFUSE_DIS_USB_SERIAL_JTAG_ROM_PRINT. The JTAG signal source is selected using GPIO3, with additional configuration options provided by the eFuses EFUSE_DIS_PAD_JTAG, EFUSE_DIS_USB_JTAG, and EFUSE_STRAP_JTAG_SEL. All of these eFuse parameters are initially set to zero, indicating they are unprogrammed. Since eFuses are one-time programmable, setting any of these bits to one is permanent and cannot be reversed.

Chip Boot Mode Control The boot mode of the ESP32-S3 is determined by the states of GPIO0 and GPIO46 after reset. If GPIO0 is high, the device enters SPI Boot mode, which is the default configuration, allowing the ROM bootloader to load and execute code directly from the SPI flash. If both GPIO0 and GPIO46 are low, the chip enters Joint Download Boot mode, which supports multiple download methods

including USB Download Boot via USB-Serial/JTAG or USB-OTG, as well as UART Download Boot. In this mode, users can transfer binary files into flash memory or SRAM using a UART or USB interface. Additionally, the ESP32-S3 supports SPI Download Boot mode as an alternative.

VDD_SPI Voltage Control The VDD_SPI voltage on the ESP32-S3 can be configured using GPIO45 and eFuse settings, depending on the value of EFUSE_VDD_SPI_FORCE. By default, the voltage is set to 3.3 V through the RTC domain. If custom voltage levels are needed, such as 1.8 V from the flash regulator, this can be enabled by setting the relevant eFuse bits. The eFuse settings EFUSE_VDD_SPI_FORCE and EFUSE_VDD_SPI_TIEH determine whether the source and level of VDD_SPI are fixed or software-configurable.

ROM Messages Printing Control During boot, ROM messages from the chip can be printed to different interfaces. By default, output is directed to the UART0 or the USB Serial/JTAG controller. This behavior can be modified by setting specific eFuse bits or adjusting registers, allowing the messages to be redirected to either UART0 or USB, depending on system needs.

JTAG Signal Source Control The JTAG signal source on the ESP32-S3 can be configured during early boot using the strapping pin GPIO3 in combination with several eFuse settings. Since GPIO3 lacks internal pull resistors, its strapping state must be externally controlled and must not be left floating. Depending on the configuration of EFUSE_DIS_PAD_JTAG, EFUSE_DIS_USB_JTAG, and EFUSE_STRAP_JTAG_SEL, JTAG signals can be routed either to the USB Serial/JTAG controller or to the dedicated JTAG pins (MTDI, MTCK, MTMS, MTDO). If required, JTAG functionality can also be completely disabled by setting the appropriate eFuses.

4.2.5. Peripherals

The ESP32-S3 features a range of integrated peripherals including interfaces such as SPI, UART, I2C, I2S, LCD, camera, and SD/MMC, as well as functional modules like LED PWM, pulse counter, remote control, and USB Serial/JTAG. The chip also supports motor control (MCPWM), analog input (ADC), capacitive touch, and temperature sensing. Additionally, it integrates a full-speed USB 2.0 OTG interface for direct USB communication and includes a TWAI® controller, which is compatible with CAN Specification 2.0 (ISO 11898-1).

Connectivity Interface

The connectivity interfaces on the chip enable communication and interaction with external devices and networks.

UART Controller Three UART controllers—UART0, UART1, and UART2—designed for high-speed, full-duplex asynchronous communication are included in the ESP32-S3. Supporting speeds up to 5 Mbps, these controllers are compatible with RS232, RS485, and IrDA protocols. Each UART shares a common 1024 x 8-bit RAM for transmit (TX) and receive (RX) FIFOs and offers features like programmable baud rate, automatic baud rate detection, special character detection (AT_CMD), hardware/software flow control, and GDMA support for high-speed data transfers. UARTs can also be used as a wake-up source from low-power states. In terms of pin assignments, UART0 and UART1 use fixed GPIOs by default but can be mapped to any GPIO via the GPIO Matrix. UART0 transmit (U0TXD) and receive (U0RXD) are multiplexed with GPIO43 and GPIO44, while flow control pins (U0RTS/U0CTS and U0DTR/U0DSR)

can also be routed through various GPIOs. Similarly, UART1 uses GPIO17 and GPIO18 for TX/RX and GPIO19 and GPIO20 for RTS/CTS, but again, all signals can be reassigned via the GPIO Matrix. UART2 offers complete flexibility, allowing all its signals to be mapped freely to any GPIO.

I2C Interface The ESP32-S3 features two I2C bus interfaces that can operate in either master or slave mode, depending on the application. It supports standard mode (100 kbit/s), fast mode (400 kbit/s), and speeds up to 800 kbit/s, limited by the strength of the pull-up resistors on the SCL and SDA lines. Both 7-bit and 10-bit addressing modes are available, with support for double addressing (slave and register addressing). The hardware includes a command abstraction layer to simplify I2C usage. I2C pins can be mapped to any GPIO via the GPIO Matrix, offering flexible pin configuration.

I2S Interface Integrated are two standard I2S interfaces that support both master and slave modes, as well as full-duplex or half-duplex communication. These interfaces are configurable, allowing for 8-, 16-, 24-, or 32-bit data resolution for both input and output channels. They support BCK clock frequencies ranging from 10 kHz to 40 MHz and include a dedicated DMA controller. The I2S interface is compatible with TDM PCM, TDM MSB/LSB alignment, TDM Phillips, and PDM formats. Like other peripherals, I2S signals can be mapped to any GPIO using the GPIO Matrix.

LCD and Camera Controller A LCD and camera controller enables efficient handling of parallel video data streams. The LCD module supports 8- to 16-bit parallel RGB, I8080, and MOTO6800 interfaces, with operating speeds up to 40 MHz. It includes built-in support for format conversion between RGB565, YUV422, YUV420, and YUV411. On the input side, the camera module is designed to interface with DVP image sensors over an 8- to 16-bit parallel bus, also running at frequencies up to 40 MHz. It offers the same range of video format conversions as the LCD module. Pin assignments for both interfaces are flexible and can be configured using the GPIO Matrix.

Serial Peripheral Interface (SPI) The ESP32-S3 provides a versatile and high-performance Serial Peripheral Interface (SPI) system consisting of four separate SPI controllers: SPI0, SPI1, SPI2, and SPI3. SPI0 and SPI1 are primarily reserved for internal operations, such as communication with flash memory and PSRAM, and are typically not used for general-purpose peripheral control. SPI2 and SPI3, on the other hand, are designed as flexible general-purpose SPI interfaces and can be configured as needed. Both SPI2 and SPI3 support single, dual, quad, and octal SPI communication, offering compatibility with a broad range of external devices. These interfaces allow operation in both master and slave modes, and support full-duplex and half-duplex data transfers. Clock frequencies can be set up to 80 MHz in SDR (single data rate) mode and up to 40 MHz in DDR (double data rate) mode, with full configurability of clock polarity (CPOL) and phase (CPHA), providing fine-grained control over signal timing. Data can be transmitted in bytes, with selectable bit order (MSB first or LSB first). Furthermore, SPI2 and SPI3 support DMA (via GDMA) for high-speed, low-latency data transfers without CPU intervention. SPI2 additionally offers connection to up to six independent SPI slaves, with configurable chip-select (CS) setup time and hold time. When acting as a master, SPI2 can support 2-, 4-, or 8-line half-duplex communication and 2-line full-duplex communication, depending on the mode and peripheral requirements. As a slave, it supports 2-line full-duplex communication up to 60 MHz, and half-duplex on 2-, 4-, or 8-line configurations. Pin assignment for each SPI controller is flexible. SPI2 and SPI3 pins can be mapped freely to any GPIO using the GPIO Matrix. SPI0 and SPI1 use multiplexed pins

selected through the IO MUX. For SPI0, the pin mapping depends on the selected interface mode. For example, using GPIO26–GPIO29 enables a 4-bit interface and supports basic SPI functions, while GPIO33–GPIO36 allow access to an 8-bit data line and dual data rate SPI. It is also possible to configure SPI0 to share pins with the JTAG interface, although this is generally not recommended unless JTAG is not required. SPI1 offers a similar level of flexibility through IO MUX or the GPIO Matrix and is typically used in scenarios where SPI0 is occupied or where different electrical characteristics are required. Both interfaces support fallback options for pin assignment to avoid conflicts with other high-priority functions such as touch sensors, ADC inputs, or USB functionality.

Two-Wire Automotive Interface (TWAI) The Two-Wire Automotive Interface (TWAI®) supports controller area network (CAN) communication based on the ISO 11898-1 protocol, commonly referred to as CAN 2.0. It enables multi-master, multicast communication with integrated mechanisms for error detection, signaling, message prioritization, and bus arbitration. TWAI supports both standard (11-bit identifier) and extended (29-bit identifier) frame formats and operates at bit rates ranging from 1 kbit/s to 1 Mbit/s. It includes a 64-byte receive FIFO, an acceptance filter supporting single and dual filtering modes, and multiple operational modes such as normal, listen-only, and self-test. Advanced error handling is provided through error counters, configurable interrupt thresholds, error code capture, and arbitration loss detection. Pin assignment is flexible and fully configurable via the GPIO Matrix.

USB 2.0 OTG Full-Speed Interface The USB OTG interface in the ESP32-S3 operates at full-speed in compliance with the USB 2.0 specification and integrates a built-in transceiver. It supports both Full-Speed (FS) and Low-Speed (LS) data rates and allows Host Negotiation Protocol (HNP) and Session Request Protocol (SRP) operation in both A-device and B-device roles. Flexible memory access modes are available, including scatter/gather DMA, buffer DMA, and slave mode. Depending on system configuration, USB Serial/JTAG and USB OTG can share the integrated transceiver through time-division multiplexing or operate independently using separate internal and external transceivers. In device mode, endpoint 0 is always available as a bidirectional control endpoint, while endpoints 1 through 6 are fully configurable as either IN or OUT. At most, five IN endpoints can be active simultaneously, each with its own dedicated TX FIFO, while all OUT endpoints share a common RX FIFO. In host mode, the controller provides eight channels, including one dedicated control pipe (using paired IN and OUT channels) that supports control transfers. The remaining seven channels are dynamically assignable as either IN or OUT and support bulk, Isochronous, and interrupt transfer types. All channels access a shared RX FIFO, a non-periodic TX FIFO, and a periodic TX FIFO, each with configurable size.
Pin assignment varies depending on whether the internal or an external PHY is used. When using the internal PHY, the USB_D+ and USB_D- lines are multiplexed with GPIO19 and GPIO20 and can conflict with interfaces such as UART1 or SAR ADC2. If an external PHY is employed, signals are routed through GPIO21 and GPIO38 to GPIO42, as well as specific JTAG-function pins including MTMS, MTDI, MTCK, and MTDO.

USB Serial/JTAG Controller The integrated USB Serial/JTAG controller provides both communication and debugging functionality through a fixed-function USB device interface. It operates as a full-speed USB device and supports two OUT endpoints and three IN endpoints in addition to the control endpoint 0, with each capable of handling data payloads of up to 64 bytes. The controller adheres to the USB CDC-ACM standard for serial communication, enabling plug-and-play compatibility with most modern operating systems. In parallel, it offers a compact JTAG interface for efficient

communication with the CPU debug core, supporting advanced development and diagnostics. The USB Serial/JTAG controller can operate using either the internal USB PHY or an external PHY, with pin assignment handled through the IO MUX or GPIO Matrix. When the internal PHY is used, USB_D+ and USB_D- are multiplexed with GPIO19 and GPIO20, which may also serve functions such as UART1 or SAR ADC2. If an external PHY is selected, the controller utilizes GPIO38 through GPIO42 and routes its signals through JTAG-assigned pins, including MTMS, MTDI, MTDO, and MTCK, along with GPIO38. Additionally, the CDC-ACM functionality enables host-controlled chip reset and entry into download mode, further extending its utility in embedded development workflows.

SD/MMC Host Controller An integrated SD/MMC host controller supports high--speed communication with external memory and storage devices. It is compatible with multiple industry standards, including SD memory versions 3.0 and 3.01, SDIO version 3.0, and CE-ATA version 1.1. Additionally, support is provided for MMC version 4.41 as well as eMMC versions 4.5 and 4.51, offering flexibility across a wide range of storage solutions. The controller can output a clock signal at frequencies up to 80 MHz and supports three data bus widths: 1-bit, 4-bit, and 8-bit. In 4-bit mode, it can operate with two SD/SDIO/MMC cards or a single SD card at 1.8 V. All related signals are assignable through the GPIO Matrix, allowing for flexible pin configuration depending on system design requirements.

LED PWM Controller A LED PWM controller enables the generation of independent digital waveforms across up to eight channels. Each channel supports configurable waveform periods and duty cycles, with a resolution of up to 14 bits achievable within a 1 ms period. The controller supports multiple clock sources, including the APB clock and the external main crystal, providing flexibility in timing configuration. It can also operate while the CPU is in light-sleep mode, ensuring uninterrupted output. For applications such as smooth color transitions in RGB lighting, the controller allows gradual adjustment of the duty cycle. Pin assignment for all PWM outputs is fully flexible via the GPIO Matrix.

Motor Control PWM (MCPWM) Two integrated MCPWM (Motor Control PWM) units provide advanced waveform generation capabilities for driving digital motors, lighting systems, and other precision-controlled devices. Each unit includes a clock prescaler, three independent PWM timers, three PWM operators, and a capture module. The timers serve as timing references, while the operators generate the output waveforms based on these references. Operators can be flexibly mapped to any of the timers, allowing for both synchronized and independent PWM signal generation. This flexible architecture enables multiple channels to share timing sources for coordinated control or operate with separate timing for individual outputs. All MCPWM output pins can be freely assigned via the GPIO Matrix.

Remote Control Peripheral (RMT) The Remote Control Peripheral (RMT) provides hardware support for sending and receiving infrared signals, commonly used in consumer remote control applications. It includes four transmission (TX) and four reception (RX) channels, all sharing a 384 x 32-bit RAM block for efficient data handling. Multiple channels can operate simultaneously, with support for pulse modulation on the transmit side and both filtering and demodulation on the receive side. The peripheral supports various operational modes, including wrap-around and continuous transmission or reception. DMA can be enabled for TX on channel 3 and RX on channel 7 to offload data movement from the CPU. All RMT signal lines can be flexibly routed through the GPIO Matrix.

Pulse Count Controller (PCNT) The pulse count controller (PCNT) offers hardware-based pulse edge counting with flexible configuration options across four independent units. Each unit supports counting values from 1 to 65.535 and contains two channels that share a single pulse counter. Input pulses and control signals are handled per channel, with built-in glitch filtering to ensure signal integrity. Channels can be configured to count on either the rising or falling edge of the input signal. Additionally, each control signal can be set to increment, decrement, or disable the counter based on its high or low logic state. All pulse and control signal pins can be freely mapped through the GPIO Matrix.

Analog Signal Processing

The ESP32-S3 provides following analog signal processing features.

SAR ADC The SAR ADC pins are multiplexed with a range of other functional interfaces, including GPIO1 to GPIO20 and their corresponding RTC functions (RTC_-GPIO1 to RTC_GPIO20). These pins are also shared with peripherals such as the touch sensor interface, SPI, UART, and the USB differential data lines (USB_D- and USB_D+). Selection between these functions is managed via the IO MUX, allowing flexible configuration depending on application requirements.

Temperature Sensor The integrated temperature sensor produces a voltage that changes proportionally with the chip's internal temperature. This voltage is converted internally into a digital value using an ADC. Operating over a range of –20 $°C$ to 110 $°C$, the sensor is intended primarily for monitoring temperature fluctuations within the chip itself. Readings are influenced by internal factors such as clock frequency and I/O activity, and typically reflect a temperature higher than the ambient environment.

Touch Sensor Fourteen GPIOs on the ESP32-S3 support capacitive touch sensing, enabling detection of changes caused by contact or proximity to a finger or other object. The system is designed with low-noise circuitry and high sensitivity, allowing for the use of small touch pads. Multiple pads can be arranged in arrays to cover larger areas or enable multi-point detection. Additional features are digital filtering and waterproofing. The assigned pins for the touch sensor are multiplexed with GPIO1 to GPIO14, RTC_GPIO1 to RTC_GPIO14, SAR ADC interface, and SPI interface via IO MUX.

Peripheral Schematics

Figure 4.13 represents a typical application circuit in which the module is connected to essential peripheral components such as the power supply, antenna, reset circuitry, JTAG interface, and UART interface.

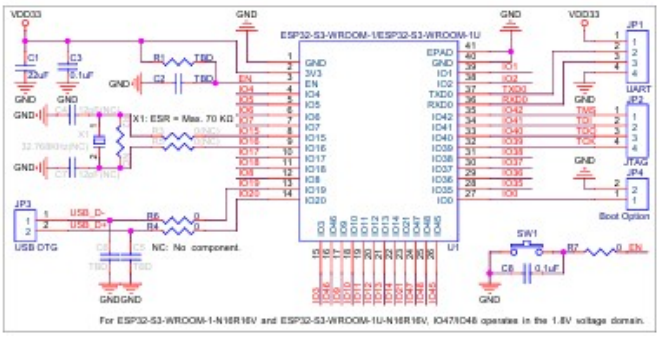


Figure 4.13.: Peripheral Schematics [see Sys24, p. 42]

4.2.6. Electrical Characteristics

The electrical characteristics define the operating limits, voltage levels, and current ratings required for reliable performance of the ESP32-S3 under various conditions.

Absolute Maximum Ratings Absolute maximum ratings define the limits beyond which permanent damage to the ESP32-S3 may occur. These ratings include the power supply voltage (VDD33), which must remain within -0.3 V to 3.6 V, and the storage temperature, limited to a range of -40°C to 105°C. These values represent stress conditions only and functional operation outside the recommended range is not guaranteed and may affect long-term device reliability.

Recommended Operating Conditions Recommended operating conditions specify the voltage, current, and temperature ranges in which the ESP32-S3 is intended to operate reliably. The supply voltage (VDD33) should remain between -0.3 V to 3.6 V, with a typical value of 3.3 V. The current (IVDD33) is defined as the current provided by the external power source. Operating temperature depends on the version: the -40°C to 85°C range applies to the standard version, while the extended temperature version supports up to 105°C. Staying within these conditions ensures optimal device performance and longevity.

DC Characteristics The DC characteristics of the ESP32-S3 define the voltage and current levels required for proper digital input and output operation at a supply voltage of 3.3 V and a temperature of 25°C. Input thresholds are specified with a minimum high-level input voltage of 0.75 × VDD and a maximum low-level input voltage of 0.25 × VDD, ensuring reliable logic detection. Input leakage currents are minimal, with typical values below 50 nA. Output levels are characterized by a high-level output voltage of at least 0.8 × VDD and a low-level output voltage not exceeding 0.1 × VDD when measured with a high-impedance load. Depending on the pad configuration, output drivers can source or sink up to 40 mA and 28 mA respectively. Internal weak pull-up and pull-down resistors are provided, with typical resistance values of 45 kΩ and 25 kΩ. Reset pin behavior is also defined, with release and assertion thresholds consistent with standard digital logic levels, ensuring reliable device startup and shutdown conditions.

Current Consumption Characteristics Current consumption characteristics for the ESP32-S3 vary depending on the operating mode, supply voltage, and system configuration. In active mode with a 3.3 V supply and ambient temperature of 25°C, peak TX currents can range from 267 mA to 355 mA depending on the 802.11 protocol and data rate, while RX current remains under 100 mA. In modem-sleep mode, current

consumption is significantly reduced, with values depending on CPU frequency, core activity, and instruction set. Dual-core operation with 32-bit instructions shows the highest values, while single-core operation in idle state is considerably more efficient. At 240 MHz, for example, dual-core execution of 128-bit instructions can draw up to 103.9 mA.
In low-power modes, such as light-sleep or deep-sleep, typical current drops to the microamp range 8 μA for deep-sleep and 7 μA for hibernation. The chip can also be fully powered off by pulling the CHIP_PU pin low, consuming just 1 μA. It's important to note that for variants with embedded PSRAM, additional current must be considered depending on the memory size and operating voltage. For example, 8 MB Octal PSRAM at 3.3 V can add around 140 μA, while 2 MB Quad PSRAM at 3.3 V adds approximately 40 μA in light-sleep mode. These measurements highlight the flexibility of the ESP32-S3's power management, making it suitable for both high-performance and energy-constrained applications.

4.2.7. RF Characteristics

RF measurements are taken at the antenna port, where the RF cable is connected, and include losses from the front-end circuitry. For modules with external antenna connectors, the test setup uses antennas with a 50Ω impedance. Devices are intended to operate within the center frequency ranges allocated by regional regulatory authorities. Both the target center frequency and the transmit power level can be configured through software, allowing adaptation to specific compliance and application requirements.

Wi-Fi Radio

Wi-Fi functionality on the ESP32-S3 is implemented according to the IEEE 802.11b/g/n standards and operates within a center frequency range of 2412 MHz to 2484 MHz. The transmit (TX) characteristics vary depending on the modulation and data rate. For example, typical transmit power reaches up to 20.5 dBm for 802.11 b at 1 Mbps, while 802.11 n using HT40 and MCS 7 operates at a typical 17.0 dBm. Error Vector Magnitude (EVM) values are also provided for each mode, with stricter limits applied to higher data rates and modulation schemes. For instance, 802.11 n HT40 at MCS 7 must meet an EVM limit of –27 dB when transmitting at 17 dBm. Receiver (RX) sensitivity is defined by a packet error rate (PER) of 8% for 802.11 b and 10% for 802.11g/n. The RX sensitivity values indicate the minimum signal level required for reliable communication. At 1 Mbps (802.11 b), the receiver can detect signals as low as -98.2 dBm, while for 802.11 n HT40 at MCS 7, typical sensitivity is -71.2 dBm. Maximum RX level values are also provided to indicate the strongest signal the receiver can tolerate without performance degradation, which remains consistently at 0 to 5 dBm across data rates. The Adjacent channel rejection values for the ESP32-S3 vary depending on the modulation and data rate. At 802.11g with 6 Mbps, the typical adjacent channel rejection is 35 dB, while at higher-throughput modes such as 802.11 n HT40 with MCS 7, the rejection decreases to around 8 dB.

Bluetooth LE Radio

Bluetooth LE functionality operates across a center frequency range of 2402 MHz to 2480 MHz, with a configurable RF transmit power ranging from -24.0 dBm up to 20.0 dBm. Transmitter characteristics are specified for various data rates including 1 Mbps, 2 Mbps, 500 Kbps, and 125 Kbps. Each configuration is defined by limits on carrier frequency offset and drift, modulation accuracy, and in-band spurious emissions. For instance, at 1 Mbps, the carrier frequency offset must not exceed $/pm$150ß,kHz and modulation deviation must remain within -198.00 kHz for 99.9% of symbols. Spurious emissions are limited to -42 dBm within $/pm$2 MHz of the carrier and -44 dBm beyond

/*pm*3 MHz.Receiver characteristics follow similar categorization per data rate. At 1 Mbps, sensitivity is defined at –96.5 dBm for a 30.8% packet error rate. Co-channel and adjacent channel rejection values are listed for different frequency offsets, enabling assessment of interference tolerance. For example, at 2 Mbps, co-channel interference rejection starts at 4 dB for F = F0 /*pm*1 MHz and increases to 8 dB for F = F0 /*pm*2 MHz. Adjacent channel selectivity and image frequency rejection are also provided, with values varying depending on signal spacing. These parameters allow evaluation of Bluetooth LE performance across different operating conditions. The available measurements collectively define transmitter accuracy, spectral containment, and receiver sensitivity—core factors that determine link stability and coexistence performance.

4.3. LSM100A

4.4. Kommunikationsmodul LSM100A

WS: gefallen bilder und quelle

Das LSM100A stellt ein ultrakompaktes Funkmodul dar, das speziell für die Anforderungen des Internet der Dinge (IoT) entwickelt wurde. Die technische Besonderheit dieses Moduls liegt in seiner Hybrid-Architektur: Es vereint zwei der maßgeblichen LP-WAN-Technologien (Low Power Wide Area Network) – LoRaWAN und Sigfox – auf einem einzigen Chip-System.

4.4.1. Technische Kernaspekte und Funktionalität

Ein zentrales Merkmal des LSM100A ist sein Dual-Mode-Betrieb. Das Modul ist in der Lage, sowohl innerhalb eines LoRaWAN-Netzwerks als auch über die Sigfox-Infrastruktur zu kommunizieren, wobei der Wechsel zwischen den Protokollen flexibel per Software oder über standardisierte AT-Befehle erfolgt. Technologisch basiert das Modul auf dem STM32WLE5-Chip von STMicroelectronics und wird in enger Kooperation mit dem Hersteller Seong Ji Industrial (SJI) gefertigt.

Die Funkkommunikation findet im Sub-GHz-Bereich zwischen 863 MHz und 928 MHz statt, was insbesondere in Europa dem lizenzfreien ISM-Band entspricht. Trotz der sehr hohen Reichweite, die im freien Feld viele Kilometer abdecken kann, arbeitet das System mit einem minimalen Stromverbrauch. Diese Effizienz prädestiniert das Modul für den Einsatz in autarken, batteriebetriebenen Geräten, bei denen eine langfristige Wartungsfreiheit im Vordergrund steht.

4.4.2. Physikalische Spezifikationen und Schnittstellen

Mit einer Grundfläche von lediglich etwa 14 mm × 15 mm weist das LSM100A eine extrem kompakte Bauform auf, die eine Integration in platzkritische Hardware-Designs ermöglicht. Für die Anbindung externer Peripherie und Sensoren stellt das Modul eine Vielzahl an Schnittstellen zur Verfügung, darunter UART, SPI, I2C sowie analoge Eingänge (ADC) und universelle digitale Ein-/Ausgänge (GPIOs).

Besonders hervorzuheben ist die Energieeffizienz im Betrieb: Im sogenannten Deep-Sleep-Modus reduziert sich die Stromaufnahme auf weniger als 2 µA, was theoretische Batterielaufzeiten von mehreren Jahren realisierbar macht. In der Standardausführung erfolgt die Steuerung des Moduls über eine serielle UART-Schnittstelle mittels einfacher AT-Befehle, was die Implementierung in bestehende Systemarchitekturen erheblich vereinfacht.

4.5. Power Management

The Hypatia Board features a USB-C interface 4.14, which serves both as the primary power supply and as a means for uploading code to the ESP32 microcontroller. In addition, it includes a JST connector for attaching a battery, as well as a separate JST input for a solar panel, enabling the integration of an auxiliary power source and the ability to recharge the battery.

4.5.1. USB-C Interface

The board is equipped with a USB-C interface, which serves as both a power input and a data connection for programming and communication with the ESP32-S3 microcontroller. Through this single, compact connector, users can supply external 5V power to the system or interface with a host computer for firmware uploads, serial debugging, or data exchange.

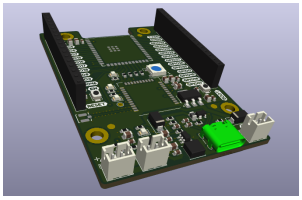


Figure 4.14.: USB-C Interface on Hypatia Board

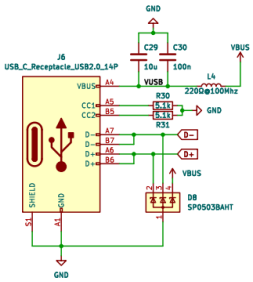


Figure 4.15.: USB-C Interface on Hypatia Board Circuit diagram

SP0503BAHT TVS diode array

To protect the system's USB interface from electrostatic discharge (ESD) and transient voltage events, the SP0503BAHT TVS diode array 4.16 is implemented between the USB-C connector and the internal circuitry. This component provides dedicated protection for the USB data lines (D+ and D-) as well as the VBUS supply line (+5V supply line in the USB port) 4.15. During events such as cable insertion or user interaction, the SP0503BAHT clamps sudden voltage spikes and diverts excess energy to ground. Its integration enhances for example the ESP32 microcontroller and other sensitive components from electrical overstress.

WS:SP0503BAHT Datasheet

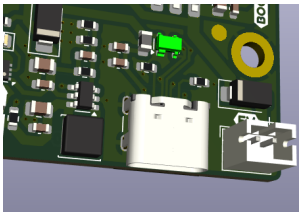


Figure 4.16.: SP0503BAHT on Hypatia Board

SS14 Diode for USB-C Protection

A SS14 Diode is integrated into the circuit to secure the power path from the USB-C input toward the charging controller. Its primary function is to prevent reverse current from flowing back into the USB power source.

WS: Hypatia Datasheet

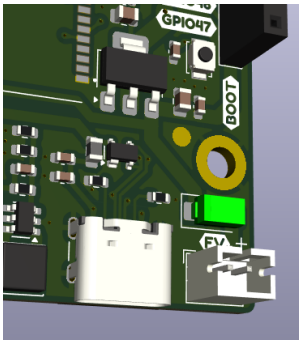


Figure 4.17.: SS14 Diode for USB-C Interface on Hypatia Board

4.5.2. Battery

An external LiPo battery can be connected to the board via a dedicated JST connector 4.18. The battery should provide a nominal output voltage of 3.7V, with a maximum fully charged voltage of 4.2V. When either a solar panel or a USB-C cable is simultaneously connected, the board is capable of recharging the battery. The battery begins to contribute meaningfully to the board's power management system when its voltage exceeds 3.0V, which serves as the effective lower operational threshold.

WS: Hypatia Datasheet

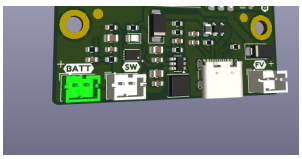


Figure 4.18.: Battery on Hypatia Board

Charger for Single-cell Li-Ion Batteries MC34673

The charging process is managed by the MC34673 4.19, a fully integrated single-cell Li-Ion/LiPo battery charger. When an external power source—such as a USB-C connection or a solar panel—is present, the MC34673 initiates a controlled multi-phase charging cycle. This includes an initial trickle charge phase for deeply discharged cells, followed by constant current (CC) charging for efficient energy delivery, and concluding with a constant voltage (CV) phase to safely reach the battery's full charge capacity at 4.2V.

The charging current is configurable via an external resistor and is automatically adjusted through internal monitoring of voltage and thermal conditions. Built-in over-voltage protection (OVP), thermal foldback, and end-of-charge detection ensure both operational safety and longevity of the battery. Furthermore, when no power source is available, the MC34673 enters a low-power standby mode, drawing less than 1μA, thus preserving battery energy. In this architecture, the MC34673 functions as the system's charging controller, coordinating power flow between sources and the energy storage component.

WS:Add Reference Datasheet

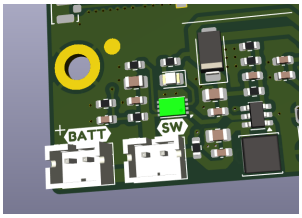


Figure 4.19.: MC34673 on Hypatia Board

Step-up Converter TLV61046ADB

A TLV61046ADB step-up converter is implemented in the circuit 4.20 to elevate the battery's variable voltage to a stable 5V output. This regulated 5V rail is essential for powering downstream components that require a constant supply, regardless of the battery's charge state.

WS: Add Reference Datasheet

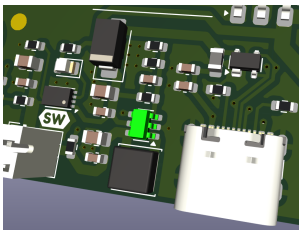


Figure 4.20.: TLV61046ADB on Hypatia Board

AMS1117-3.3 linear voltage regulator

To supply components that operate at lower voltages, the board integrates an AMS1117-3.3 linear voltage regulator 4.21, which derives its input from the 5V rail generated by the TLV61046ADB boost converter. The AMS1117-3.3 provides a regulated 3.3V output, which is required by the ESP32-S3 microcontroller and other low-voltage peripherals.

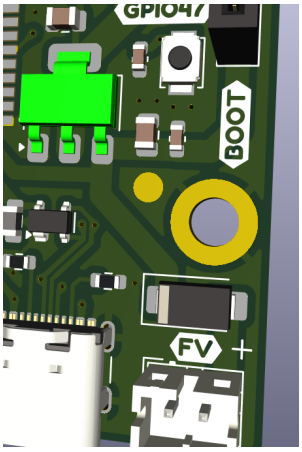


Figure 4.21.: AMS1117 on Hypatia Board

Read battery Voltage Code Example

A resistive voltage divider is implemented to monitor the battery voltage. The divider scales the voltage from the +BATT line down by a factor of 2, making it accessible through GPIO10.

```
const int pinVoltage = 10; // Pin connected to the voltage divider
float batteryVoltage = 0.0; // Variable to store the battery voltage

void setup() {
Serial.begin(9600); // Start serial communication
}

void loop() {
int rawValue = analogRead(pinVoltage); // Read the analog value
batteryVoltage = (rawValue * (5.0 / 1023.0)) * 2; // Calculate battery voltage
Serial.print("Battery Voltage: ");
Serial.print(batteryVoltage);
Serial.println(" V");
delay(1000); // Wait 1 second
}
```

Listing 4.2: WiFi Check LED Code

4.5.3. Solar panel

An additional JST connector 4.22 on the board allows for the integration of a solar panel as an alternative power source. This enables the system to harvest energy from ambient light, providing a supplementary supply to power the board and recharge the connected battery.

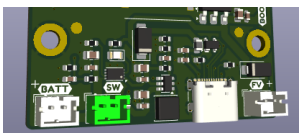


Figure 4.22.: Solar Panel JST on Hypatia Board

SS14 Diode as Solar Panel Protection

A SS14 diode is placed between the solar panel input and the charging circuit to ensure unidirectional current flow. This diode prevents reverse current from flowing

back into the solar panel when it is inactive or exposed to low light conditions, thereby protecting the panel and enhancing the overall efficiency and reliability of the power management system.

WS: Add Reference Datasheet

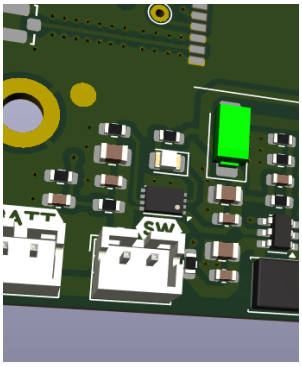


Figure 4.23.: SS14 on Hypatia Board for Solar Panel

5. Hardware XY

6. Sensor XY

Introduction

WS:cite books, applications, board

6.1. General

General description
cite books

6.2. Specific Sensor/Actor

cite board

6.3. Specification

- Cite the data sheet
- Extract te information from the data sheet
- Circuit Diagram

6.4. Library

6.4.1. Description

6.4.2. Installation

- data types
- data structure
- data quantity
- data quality

6.4.3. Functions

6.5. Example

In this section, a simple example is described in detail, which demonstrates how the library supports the sensor/actor.

6.5.1. Manual

6.5.2. Inside the Example

6.5.3. Code

6.5.4. Files

6.6. Calibration

cite method

6.7. Simple Code

6.8. Simple Application

6.9. Tests

6.9.1. Simple Function Test

6.9.2. Test all Functions

6.10. Further Readings

7. Actor XY

Introduction

WS:cite books, applications, board

7.1. General

General description
cite books

7.2. Specific Sensor/Actor

cite board

7.3. Specification

- Cite the data sheet
- Extract te information from the data sheet
- Circuit Diagram

7.4. Library

7.4.1. Description

7.4.2. Installation

- data types
- data structure
- data quantity
- data quality

7.4.3. Functions

7.5. Example

In this section, a simple example is described in detail, which demonstrates how the library supports the sensor/actor.

7.5.1. Manual

7.5.2. Inside the Example

7.5.3. Code

7.5.4. Files

7.6. Calibration

cite method

7.7. Simple Code

7.8. Simple Application

7.9. Tests

7.9.1. Simple Function Test

7.9.2. Test all Functions

7.10. Further Readings

Part III.

Domain Knowledge - Application

8. Application

Part IV.

Domain Knowledge - Tools

9. Python Tool XY

10. Hardware Tool XY

Part V.

Developmenmt

11. Flow Chart Development XY

12. Database XY

13. Data Selection XY

14. Data Transformation XY

15. Data Mining XY

Part VI.

Development to Deployment

16. Machine Learning Pipeline XY

17. Database XY

Part VII.

Verification - Evaluation - Conclusion

18. Evaluation/Verification XY

19. To DoX Y

20. Conclusion XY

Part VIII.

Appendix

21. Bill of Material

22. SBOM

requirements.txt

23. Package Example

23.1. Introduction

23.2. Description

23.3. Installation

```
pip packageExample
```

23.4. Example - Manual

23.5. Example

23.6. Example - Code

23.7. Example - Files

PackageExample.py

23.8. Further Reading

References

[Aba+16] M. Abadi et al. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

Part IX.

Hints - Examples for LaTeX - Read and Use

24. Hints

Please, use **biber.exe**.

If you want to create a symbol directory, call makeindex:
makeindex %.nlo -s nomencl.ist

24.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier curves are formed with the help of Bernstein polynomials. If at least two points and two tangents are known, control points can be determined with which a control polygon is formed. The course of the curve is oriented to this control polygon. The number of control points depends on the degree of the Bézier curve. A Bézier curve of degree n has n+1 control points. The Bézier curve is calculated using the De Casteljau algorithm.

Definition. In the interval [0; 1] the **Bernstein polynomial of n$^{\text{th}}$ degree** is defined by:

$$b_{i,n}(\lambda) = \binom{n}{i}(1-\lambda)^{n-i}\lambda^i, \quad \lambda \in [0,1], \quad i = 0, \ldots, n \tag{24.1}$$

Definition. The binomial coefficient is defined by:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \ldots, n \tag{24.2}$$

Here the Bernstein polynomials $b_{i,n}$ form a basis of the vector space $\mathbb{P}^n(I)$ of polynomials of degree at most n over I. Thus every polynomial of degree at most n can be written uniquely as a linear combination. [Far02]

Beispiel. Determination of Bernstein polynomials for $n = 3$ holds:

$$\begin{aligned}
b_{3,0}(\lambda) &= \binom{3}{0}(1-\lambda)^{3-0}\lambda^0 &&= (1-\lambda)^3 \\
b_{3,1}(\lambda) &= \binom{3}{1}(1-\lambda)^{3-1}\lambda^1 &&= 3\lambda(1-\lambda)^2 \\
b_{3,2}(\lambda) &= \binom{3}{2}(1-\lambda)^{3-2}\lambda^2 &&= 3\lambda^2(1-\lambda) \\
b_{3,3}(\lambda) &= \binom{3}{3}(1-\lambda)^{3-3}\lambda^3 &&= \lambda^3
\end{aligned}$$

$b_{3,i}(\lambda)$ with $i = 0, 1, 2, 3$ are the cubic Bernstein polynomials of degree 3.

Definition. Given are the points

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \ldots, n$$

A **Bézier curve** is then defined by

$$C(\lambda) = \sum_{i=0}^{n} Q_i \cdot b_{i,n}(\lambda) \tag{24.3}$$

The points $Q_i, i = 0, \ldots, n$ are called **control points**.

The control points of a Bézier curve form the so-called control polygon.

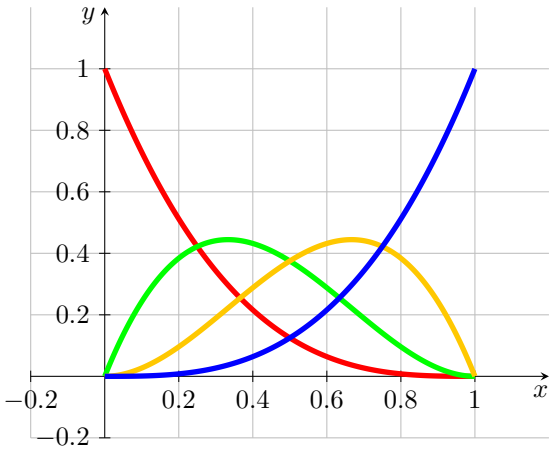


Figure 24.1.: Bernstein polynomial of degree 3

Bemerkung. Let the starting point P_0 and the end point P_1, as well as the tangents $\vec{t}_0$ and $\vec{t}_1$ be given. The tangents are not necessarily normalised.
The control points Q_0, Q_1, Q_2 and Q_3 of the associated Bézier curve are given by the following equations:

$$Q_0 = P_0, \qquad Q_1 = P_0 + \lambda_0 \vec{t}_0, \qquad Q_2 = P_1 - \lambda_1 \vec{t}_n, \qquad Q_3 = P_1 \tag{24.4}$$

[Jak+10]

Beispiel. Given are

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t}_1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Then, according to theorem **??** for control points of the associated Bézier curve results:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \qquad Q_1 = P_0 + \vec{t}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t}_n = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \qquad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

The Bézier curve and its control points are shown in the figure **??**.

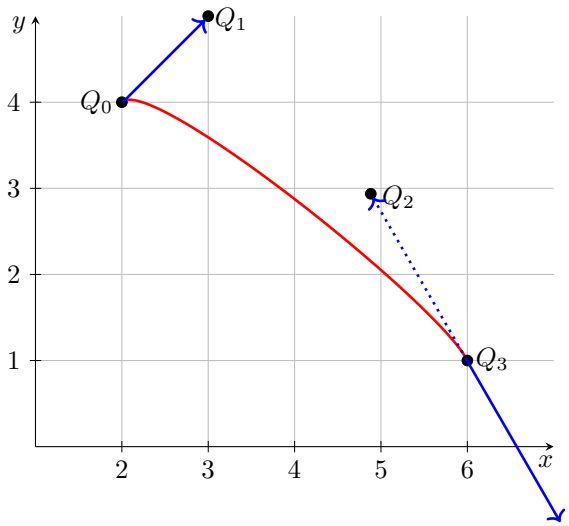


Figure 24.2.: Bézier curve for example 24.1

25. Verrundung mit einem Kreisbogen

25.1. Gleichungen

Blending with an arc is a simple variant of smoothing corners. This variant enables the processing and the generation of files according to DIN 66025. For smoothing, three points P_0 and S and P_1 are always considered, thus a symmetric Hermite problem results. According to theorem **??** it is assumed to be in the standard form $HP(L, \alpha)$.

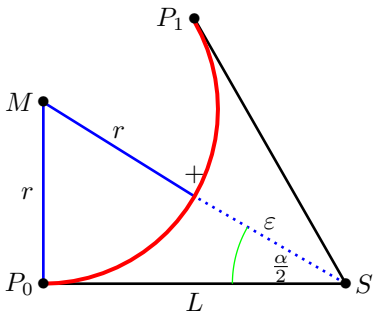


Figure 25.1.: Smoothing a corner with the help of an arc - triangle

The figure **??** represents the situation. The points P_0, S and P_1 are given. The included angle $\alpha = \angle(P_0; S; P_1)$ as well as the distance $L = |||S - P_0||| = |||P_1 - S|||$ are shown in the graph. The red arc is the desired result. The distance of its centre M to S is then $r + \varepsilon$, where r is the radius of the arc and ε is the given tolerance. Thus we obtain a right triangle P_0SM whose edge lengths are L, $r + \varepsilon$ and r.

According to the definition of the sine and the tangent, it follows:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r+\varepsilon} \qquad \text{und} \qquad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \quad \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \qquad \text{und} \qquad r = \cot\left(\frac{\alpha}{2}\right) L$$

The two equations can be combined to give the following conditions:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \qquad \text{bzw.} \qquad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

This gives the equation for the radius:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1-\cos(\alpha)}{1+\cos(\alpha}} = L \cdot \frac{\sin(\alpha)}{1+\cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

The factor for converting L and ε can be simplified using trigonometric transformations.

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1-\sqrt{\frac{1-\cos(\alpha)}{2}}}{\sqrt{\frac{1+\cos(\alpha)}{2}}} = \frac{\sqrt{2}-\sqrt{1-\cos(\alpha)}}{\sqrt{1+\cos(\alpha)}} = \frac{\sqrt{1+\cos(\alpha)}-\sin(\alpha)}{1+\cos(\alpha)}$$

By comparing this with the symmetric Hermite problem in standard form, the following notation is then obtained:

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}}$$

The previous considerations are now summarised in the following sentence.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

with $L \in \mathbb{R}^{>0}$ uad $\alpha \in (-\pi;\, \pi]$.

Then an arc can be found that connects the points P_0 and P_1, has the same tangent directions at the two points.

For the circular arcs applies:

$$r = L\cdot\left|\frac{P_{1,x}}{P_{1,y}}\right|; \quad \phi_0 = -\operatorname{sign}(\alpha)\cdot\frac{\pi}{2}; \quad \phi_1-\phi_0 = \operatorname{sign}(\alpha)\cdot\pi-\alpha = \beta;$$

$$M = P_0 + \operatorname{sign}(\alpha)\cdot r\cdot\vec{t}_0^{\perp} = \operatorname{sign}(\alpha)\cdot r\cdot\begin{pmatrix}0\\1\end{pmatrix}$$

From the specification of the distance L, the maximum error can be calculated, as shown in theorem **??**. According to the derivation, it is also possible to specify the maximum error ε and determine the maximum distance L from it.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

with $L \in \mathbb{R}^{>0}$ and $\alpha \in (-\pi;\, \pi]$.

a) Let the maximum error ε be given. The maximum distance L for which an arc exists according to the theorem **??** that takes the error into account is given by:

$$L(\varepsilon,\alpha) = \varepsilon\cdot\frac{P_{1,x}}{\sqrt{P_{1,x}}-P_{1,y}} = \varepsilon\cdot\frac{L+L\cdot\cos(\alpha)}{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}$$

b) When the distance L is specified, the following maximum error results:

$$\varepsilon(L,\alpha) = L\cdot\frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}} = L\cdot\frac{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}{L+L\cdot\cos(\alpha)}$$

These considerations result in limitations for the use of this strategy, which are summarised in the following comment.

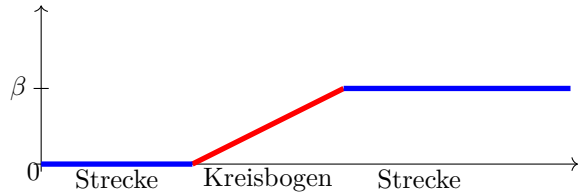


Figure 25.2.: Angular change with blending arc

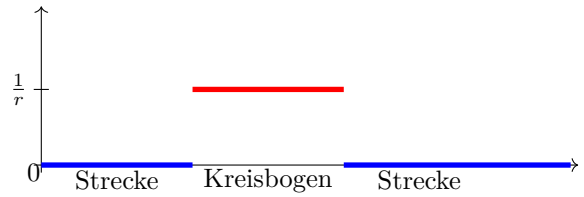


Figure 25.3.: Curvature progression for a blending arc

Bemerkung. The following conditions for applying the strategy of a circle must be fulfilled:

a)
$$P_0 \neq P_1$$

b)
$$\vec{t_0} = -\vec{t_1} \quad \Leftrightarrow \quad \alpha = 0$$

c)
$$\vec{t_0} = \vec{t_1} \quad \Leftrightarrow \quad \alpha = \pi$$

25.2. Bewertung

Rounding by means of a circular arc leads to a continuous course of the angle change. The figure **??** illustrates this.

Due to the rounding with a circular arc, the curvature of the curve is not continuous. The figure **??** shows that the curvature is constant in the range of distances 0 and on the circular arc with the value $\frac{1}{r}$, where r is the radius of the circular arc used. Due to the formula $a = \frac{v^2}{r}$ for a movement on a circular arc, the acceleration course exhibits continuity jumps at the transitions for a constant path velocity.

Depending on the angle change β at the corner S and the tolerance ε, the maximum length of the shortening of the lines can be calculated. The figure **??** shows the corresponding diagram, where the tolerance ε is set to the value 1.

The area provided can also be specified. Depending on the distance L from the corner point S, the deviation ε can be calculated. The figure **??** represents the function as a function of L and the angle α.
The figures **??** and **??** show the curves of the length as a function of the angle change for a fixed tolerance and the error as a function of the angle change for a given length. If the angle change is minimal, then the error or length L is extreme. In this case,

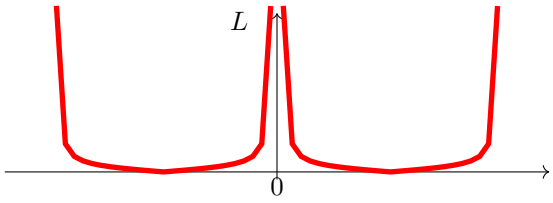


Figure 25.4.: Maximum range for a blending arc of a circle - $L(\varepsilon = const, \alpha)$

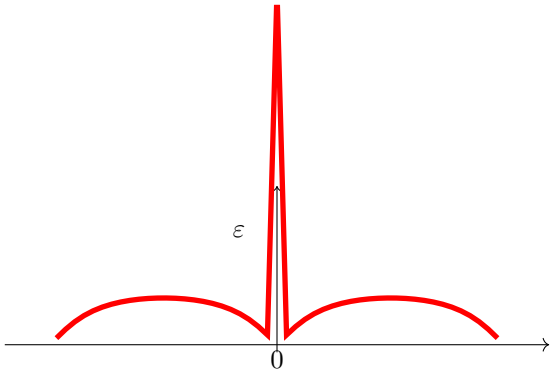


Figure 25.5.: Deviation when specifying the distance L when rounding with a circular arc

rounding by means of an arc is not possible. In order to develop a sensible strategy, a minimum angle change must be specified from which a blemnding arc is permitted. Likewise, a maximum angle change must also be defined. For an angular change of $\pm\pi$ is a bend. In this case, a blending is also not possible.

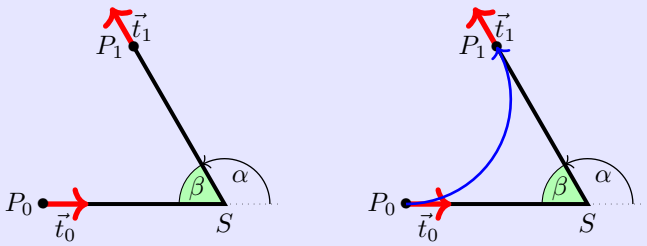
25.3. Examples

Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{2}{3}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$

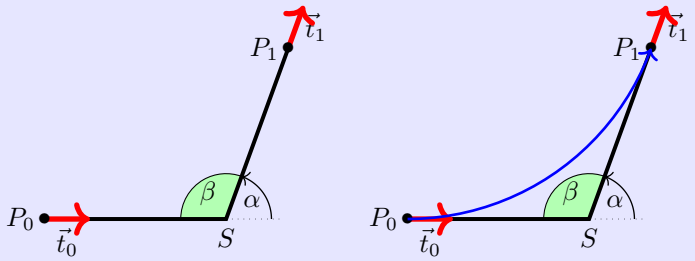


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{7}{18}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$

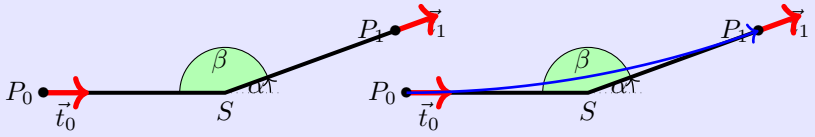


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{1}{9}\pi$.
For the blending arc follows:

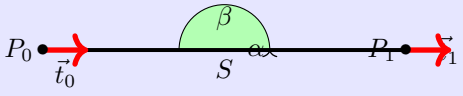
$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$



Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = 0$.
There is no blending arc here.

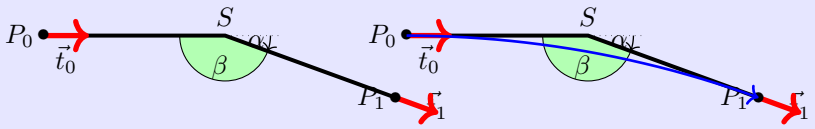


Beispiel. Let it be the symmetric Hermite problem

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}6.0+6.0\cdot\cos\left(-\frac{1}{9}\pi\right)\\6.0\cdot\sin\left(-\frac{1}{9}\pi\right)\end{pmatrix}, \begin{pmatrix}\cos\left(-\frac{1}{9}\pi\right)\\\sin\left(-\frac{1}{9}\pi\right)\end{pmatrix}, \begin{pmatrix}6.0\\0\end{pmatrix}, 6.0\right\}$$

with $L = 6$ and $\alpha = -\frac{1}{9}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix}0\\-34,0277\end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

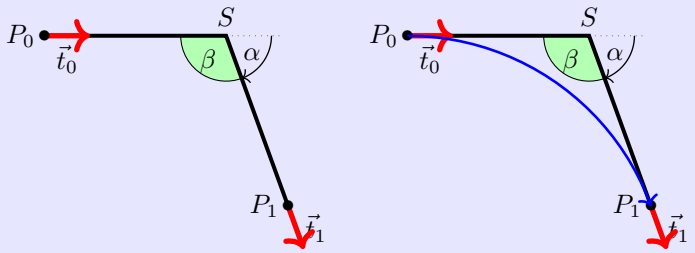


Beispiel. Let it be the symmetric Hermite problem

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}6.0+6.0\cdot\cos\left(-\frac{7}{18}\pi\right)\\6.0\cdot\sin\left(-\frac{7}{18}\pi\right)\end{pmatrix}, \begin{pmatrix}\cos\left(-\frac{7}{18}\pi\right)\\\sin\left(-\frac{7}{18}\pi\right)\end{pmatrix}, \begin{pmatrix}6.0\\0\end{pmatrix}, 6.0\right\}$$

with $L = 6$ and $\alpha = -\frac{7}{18}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix}0\\-8,5689\end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$

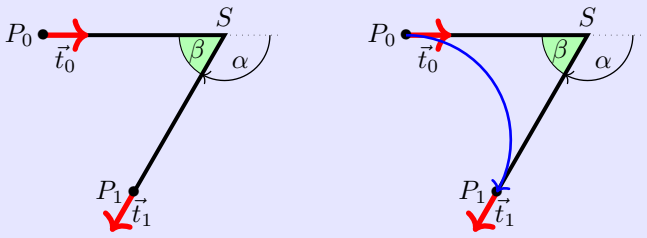


Beispiel. Let it be the symmetric Hermite problem

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}6.0+6.0\cdot\cos\left(-\frac{2}{3}\pi\right)\\6.0\cdot\sin\left(-\frac{2}{3}\pi\right)\end{pmatrix}, \begin{pmatrix}\cos\left(-\frac{2}{3}\pi\right)\\\sin\left(-\frac{2}{3}\pi\right)\end{pmatrix}, \begin{pmatrix}6.0\\0\end{pmatrix}, 6.0\right\}$$

with $L = 6$ and $\alpha = -\frac{2}{3}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix}0\\-3,4641\end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



26. Maple files

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

26.1. Geometries with Maple

The algorithms presented can be well represented using geometries. Two aspects can be investigated. On the one hand, the effort for an implementation, the computational accuracy and the stability of an algorithm are interesting; on the other hand, the methods are to be evaluated and compared with regard to their usefulness. For this purpose, modules for the formula manipulation system Maple [Wat17c] were developed. Maple offers the environment to implement algorithms easily and quickly. Furthermore, graphical representation is possible with simple means. However, a simple workbook of Maple is not designed for very large software projects. Here, one must resort to the possibility of creating and using libraries. On the one hand, Maple offers the possibility of creating one's own libraries, so-called modules. In this way, one remains within the environment and syntax of Maple. This path will be pursued further here. Another possibility is the use of libraries created by means of a higher programming language, e.g. C++, the so-called DLLs.
In the following, the creation and use of a test environment with the help of modules is described. First, the geometry elements that are stored in various modules and their use are described. The structure and use of a module in Maple is then explained so that own extensions and additions are possible.

26.2. Geometry elements used

This is a test environment for the algorithms presented, only geometries that have been described are also used.

- points (`MPoint`)
- lines (`MLine`)
- arcs (`MArc`)
- Bézier curves (`Bezier`)
- polygon courses (`MPolygon`)
- Geometry List (`MGeoList`)
- Hermite problems (`MHermiteProblem`)
- Symmetric Hermite Problems (`MHermiteProblemSym`)

For each geometry element a corresponding module has been created, the name of which is given in the list above. The list of which functions are available is presented in another section.
When implementing such a project, it quickly becomes clear that when using several geometry elements, a clear data structure and well-defined access to the data is essential. For example, when representing straight lines, the decision must be made

whether the representation should be point-directional or by means of two points. The choice is generally made depending on the application. Here, the path is taken that, due to a well-defined access to the data, the use of both representations is possible.

26.3. Building the data structure

The idea is to use a data structure that is as uniform and simple as possible. Maple does offer the possibility to define objects. However, it is difficult to use within a procedural environment. Therefore, all data is basically represented as lists. The first list element always contains an identifier of the element **??**. This is followed by the data, which in turn can be geometry elements. It should be noted that direct access to the data cannot be prevented; here the user is responsible.

Access to the data should be exclusively via procedures of a module. The following is an example of the data structure for points:

```
[MVPOINT,[x,y]]
```

The creation of a point then takes place via a procedure `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

When called up in the test file, the following is then output:

```
TestP0 := ["Point",[10,15]]
```

These data structures are used for the calculation of geometries. They are used in functions or procedures. Unlike variables, the data structures and procedures are defined globally and not locally. Under the command `export` the procedures are declared at the beginning of the module and can thus also be used outside the module. If, for example, a data structure from `MPoint` is to be used in another module or outside the modules, it is called as follows:

```
P0 := MPoint:-New(x,y);
```

In addition to the modules for the geometries, there is a module (MConstant) for saving constants and names. There, for `MVPOINT`, for example. „Point “ or e.g. a constant for the comparison to zero for real numbers.
Finally there is the test file, which is not a module, in which the modules are loaded, called and tested in their function. More about this file later in „1.4 Maple test file “.

26.4. Structure of a module

The following section deals with the purpose of modules and their structure.

The project is about capturing the above geometries in a data structure and representing them in Maple. However, the calculations and formulas that have to be used are a hindrance to the final representation. Modules are very well suited for summarising and hiding the calculations that take place in functions. This way, the important functions can be accessed in Maple without seeing what is in them.

Example:
The function `Angle` from the module `MPoint`: Function to calculate an angle between location vector and x-axis:

```
Angle := proc(P)
   local alpha, x, y;
```

```
   x := GetX(P);
   y := GetY(P);
   alpha := 0;
   if abs(x) < 0.00001
      then
         alpha := 3*Pi/2;
      else
         alpha := Pi/2;
      end if;
   else
      alpha := arctan(y/x);
      if x < 0
      then
         alpha := alpha + Pi;
      else
         if y < 0
         then
            alpha := alpha + 2*Pi;
         end if;
      end if;
   end if;
   return factor(alpha);
end proc;
```

This function is long, but it only represents a small part of the programme for the representation in question. That is why it is in the module and can thus be called externally with a single command:

```
Winkel := MPoint:-Angle(P)
```

The following section describes the structure of a module. Since Maple offers a wide range of possibilities, a restriction is made. Only the structure of the modules used is described, here on the basis of the module`MPoint`. For more detailed possibilities, please refer to the Maple manual [Wat17c].

structure:

The module must first be started. This is done with the name (here always a capital M and the geometry) of the module and the following command:

```
MPoint := module()
```

Then, similar to procedures, the variables and functions must be defined. You can declare them either locally or globally. If the variables or procedures are only used and changed in the module, the declaration is made with the command `local` as follows:

```
 local  Variable names, with, comma, separated;
```

If the variables or procedures are also to be usable outside the module, this is defined with `export`:

```
export Function names, with, comma, separated;
```

This is followed by the specification of the options:
`option package;`,
the description of the module:

```
description “Self-selected module description, e.g.  module for points
“;
```

and the initialisation of the module:

```
ModuleLoad := proc
  MVPOINT:=MConstant:-GetPoint();
  print("Module MPoint is loaded");
end proc;
```

From here on, programming is done as usual in Maple. The previously defined procedure and variable names are used and programming is done with commands that are also used outside of a module in Maple. It is important to know that with modules, each function can be called constantly, so the order of the functions is irrelevant.

To finally end the module, use the following command:

```
end module;
```

26.4.1. General structure of a module

In summary, the modules have the following structure:

```
Modulname := module()

   local Names, with, comma, separated;

   export Names, with, comma, separated;

   global Names, with, comma, separated;[1]

   option package;

   description „ Self-selected module description“;


   ModuleLoad := proc()[2]
      MVPOINT:=MConstant:-GetPoint();
      print("Modul MPoint is loaded");
   end proc;

 Procedure1 := proc (Passing parameters)
      inhalt;
   end proc;

   Procedure2 := proc (Passing parameters)
      content;
   end proc;


   ...

end module;
```

[1] Possible, but not used in the modules
[2] Example `MPoint`

26.4.2. Saving a module

The management of modules is done automatically by Maple. However, since the modules are passed on and have to be edited individually, some settings have to be made. Therefore, the following prefix is used for each Maple file of the project:

```
1 restart;
2 with(LibraryTools);
3 lib := "C:/FH/Tools/Maple/MyLibs/Blending.mla";
4 march('open', lib);
5 ThisModule := 'MArc';
```

The first line initialises the system. The next 3 lines enable working with archives. In the second line, the tools are loaded so that the variable `lib` can be occupied. All modules are stored in an archive; in this example it is the file `Blending.mla` in the directory `C:/FH/Tools/Maple/MyLibs/`. The command `ThisModule := 'MArc';` contains the name of the current module.

A module is then saved using the command `savelib('ModuleName')`. A file `ModuleName.mla` is then created. Depending on the configuration of Maple, the set path where the file is automatically created may not be writable. Then you can configure the system so that the mla file is stored in the current directory or in a directory of your choice.

If the above prefix is used for a Maple file, all that is required to save the module is `savelib(ThisModule, lib);`

26.4.3. Verwendung eines Moduls

A module that has been saved can now be used in other Maple worksheets. To do this, the command
`with(ModuleName)`
is used. If you have used a special path, Maple cannot find the file `ModuleName.mla`. Then the path must be made known, e.g.:
`savelibname := "c:/Maple/MyLibs";", savelibname;`
Now the procedures of the module can be accessed. An overview of all exported procedures is provided by the command
`Describe(ModuleName)`
is displayed. A list of the names including the names of the transfer parameters is displayed. If a procedure is equipped with the field `description`, this text is also displayed.

26.4.4. Creating a Maple module

Creating your own module is done quickly if you use the framework above. However, one should make some considerations beforehand and maintain a standard.

Before creating an own module, the data model and the corresponding procedures should be worked out. A programme flow chart is useful here. The central task of the module is usually quickly determined. In addition, however, the following rules should be observed.

Rule 1 Basically, both the module and each procedure receive a description. This is not limited to the optional argument `description`, but is placed in front of each procedure. The description basically contains the description of the task. The prerequisite is also mentioned or an error handling is described. Then follows the description of all input parameters, their function and their data structure. The return value is then described.

rule 2 Each module receives a procedure `Version()`, which returns the current version number.

rule 3 Data is not global. To access data, procedures `Set*` and `Get*` are provided. These procedures are also used within the procedures of a module.

rule 4 At least one test function is written for each procedure. The test function can be used to illustrate the use and any special features.

26.5. Functions of the modules

In the following, the individual modules are listed. The data structure of each module and all functions with associated tasks are listed.

26.5.1. Module `MPoint`

`MPoint` is a module for points and works with a data structure and with procedures/functions. The data structure `New` for a point is a list, which is structured as follows:

```
[MVPOINT, [x,y]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Point" and the elements are the x and y coordinates for a point. No distinction is made here between point and vector. A point can also be seen as a vector from the coordinate origin to the point.
Via the Get functions `GetX` and `GetY` one can capture the coordinates of the point and use them in other functions. The other functions can then be used to calculate with the points/vectors.

MPoint		
E	**New**	Data structure for a point
E	**GetX**	Reading the x-coordinate
E	**GetY**	reading the y-coordinate
E	**Angle**	calculating the angle between the x-axis and the point
E	**Add**	Calculates a linear combination of two points/vectors
E	**Sub**	Calculates the difference of two points/vectors
E	**Cos**	Calculates the cosine between two vectors
E	**Sin**	Calculates the sine between two vectors
E	**Scale**	Scales a vector with a factor
E	**Perp**	Calculates a vector that is orthogonal to the given vector
L	**IllustrateXY**	Plot function to illustrate a blue point
E	**Illustrate**	Plot of a blue point
E	**Plot**	Plot of a green point
E	**Plot2D**	Plot of a point with own options
E	**Length**	Calculates the distance of the point to the coordinate origin
E	**Uniform**	Normalises the vector to length 1
E	**LinetoVector**	Calculates vector from a distance
E	**Distance**	Calculates the distance between two points

26.5.2. Module MLine

MLine is a module for lines and works with a data structure and with procedures/functions. The data structure New for a line is a list which is structured as follows:

```
[MVLINE, [P0,P1]]
```

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „Line“ and the elements are the start and end points for a line.
The data structure NewPointVerctor is also a data structure for a line, but here the line is defined by a start point and a direction vector.
The Get functions StartPoint and EndPoint can be used to capture the start and end points of the route and use them in other functions. The other functions can then be used to calculate with the routes.

MLine

E	**New**	Data structure for a line (two-point form)
E	**NewPointVector**	creation of a route (point-direction form)
E	**StartPoint**	reading the starting point
E	**EndPoint**	reading the end point
E	**Position**	Calculate a point that lies on the line
E	**Plot2D**	Plot a part of the route starting from the starting point
E	**Plot2DTangent**	Plot of a point with tangent
E	**Plot2DTangentArrow**	Plot of a point with tangent (as arrow)
E	**LineLine**	Calculation of the intersection of two straight lines.
E	**AngleLine**	Calculation of the angle between two straight lines

26.5.3. Module MArc

MArc is a module for circular arcs and works with a data structure and with procedures/functions. The data structure New for an arc is a list that is structured as follows:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Arc" and the elements are the x- and y-coordinate for the centre, the radius, the start angle and the angle change for an arc.
The Get functions GetM, GetMX, GetMY, GetR, GetPhi, and GetAlpha can be used to collect the data for an arc and use it in other functions. The other functions can then be used to calculate with the arcs.

MArc

E	**New**	Data structure for an arc
E	**GetMX**	Reading the x-coordinate of the centre point.
E	**GetMY**	read the y-coordinate of the centre point
E	**GetR**	read the radius
E	**GetPhi**	reading the start angle
E	**GetAlpha**	Reading the change of angle
E	**GetM**	reading the centre point
E	**Position**	Calculating a point on the arc
E	**Plot2D**	Plot a part of the arc starting from the start angle
E	**Blend**	calculating an arc from a symmetric Hermite problem

26.5.4. module MBezier

The New data structure for a polygon is a list constructed as follows:

```
[MVBEZIER, [PointList]]
```

Within the module MBezier exist the procedures listed in the following table.

MBezier		
E	New	Manual input of control points for a Bézier curve
E	Version	Output of the verions
E	BlendCurvature	Determination of control points from symmetric Hermite problem
E	BlendCurvatureEpsilon	determination of control points from symmetric Hermite problem with given error
E	Position	position on the Bézier curve
E	GetTheta	Reading the angle
E	GetEpsilon	reading the tolerance
E	GetControlPoint	Read the control points
E	Plot2D	Read the Bézier curve
E	PlotControlPoints	representation of all control points

26.5.5. Module MPolygon

MPolygon is a module for polygons and works with a data structure and with procedures/functions. The data structure New for a polygon is a list that is structured as follows:

```
[MVPOLYGON, [PointList]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Polygon" and the elements are any number of points.
Using the Get functions GetPoint and GetN you can get the number of points and the points themselves and use them in other functions. The other functions can then be used to calculate with the points or the polygon course.

MPolygon		
E	New	Data structure for a list of points
E	GetPoint	Reading the ith point from the point list
E	GetN	Determine the number of points in the point list
E	Length	Determination of the Euclidean length of the polygon
E	Position	Calculation of a point on the polygon course
E	Tangents	calculation of the tangent
L	Plot2DAll	Plot list of all points
E	Plot2D	Plot of all points as polygonal plots.
E	Plot2DTangent	representation of the polygon course with tangent
E	PlotPoints	representation of all points

26.5.6. module MGeoList

MeoList is a module for a geometry list and works with a data structure and with procedures/functions. The data structure New for a geometry list is a list that is structured as follows:

```
[MVGEOLIST, []]
```

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „GeoList“ and the elements are any number of individual geometries.
Using the Get functions `GeoGeo` and `GetN`, the i-th element of the list and the number of elements in the list can be captured and used in other functions. The other functions can then be used to calculate with the geometries or the list. In the functions `Length`, `Plot2DAll`, `Plot2D` and `Position` the functions are called in themselves. This is possible because the functions work independently of each other.

MGeoList

E	**New**	Data structure for a geometry list
E	**Append**	Append a geometry element to the data structure
E	**Prepend**	Inserting a geometry element as the first element of the list
E	**Replace**	Replace a geometry element with another one.
E	**GetN**	Determine the number of geometry elements in the list
E	**GeoGeo**	Reading the i-th geometry element
E	**Length**	Calculate the Euclidean length of the geometry elements
E	**Position**	Calculates a point on the geometry
L	**Plot2DAll**	Plot function for plotting the geometry elements
E	**Plot2D**	Plot a part of the geometry list starting from the first element

26.5.7. Module MHermiteProblem

`MHermiteProblem` is a module for Hermite problems and works with a data structure and with procedures/functions. The data structure `New` for a route is a list, which is structured as follows:

```
[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „HermiteProb“ and the elements are two points with associated tangents (lines).
Using the Get functions `StartPoint`, `EndPoint`, `StartTangent` and `EndTangent`, the points and their tangents can be captured and used in other functions. The other functions can then be used to calculate with the data for the Hermite problem.

MHermiteProblem

E	**New**	Data structure for a Hermite problem
E	**StartPoint**	reading the start point
E	**EndPoint**	Reading the end point command
E	**StartTangent**	reading the start tangent
E	**EndTangent**	Reading the end tangent
E	**Plot2D**	Plotting the Hermite problem

26.5.8. Module MHermiteProblemSym

`MHermiteProblemSym` is a module for symmetric Hermite problems and works with a data structure and with procedures/functions. The data structure `New` for a symmetric Hermite problem is a list built as follows:

```
[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „SymHermiteProb“

and the elements are two points with associated tangents, their intersection, and the distance of the points to the intersection.

Via the Get functions StartPoint, EndPoint, StartTangent, EndTangent, CrossPoint and ParameterL one can acquire the data and use it in other functions. The other functions can then be used to calculate with the data for the symmetric Hermite problem.

MHermiteProblemSym

E	New	Data structure for a symmetric Hermite problem
E	StartPoint	reading the start point
E	EndPoint	Reading the end point command
E	StartTangent	reading the start tangent
E	EndTangent	Reading the end tangent
E	ParameterL	reading of the distance
E	CrossPoint	reading of the intersection point
E	Plot2D	Plotting of the symmetrical Hermite problem
E	Create	creation of a Sym. Hermite problem by 3 points
E	BlendArc	rounding of the corner point by an arc

26.5.9. Module MConstant

MConstant is a module for storing fixed constants and names to identify data structures. In the locally declared functions, starting with CV, the constants for declaring the different geometries are defined. These are names for recognising the geometry elements in the test file. In the globally declared functions, starting with Get, the names for identifying the geometry elements are returned.

MConstant

L	NULLEPS	constant to compare to zero
L	CVPOINT	constant for geometry elements: Point
L	CVLINE	constant for geometry elements: Line
L	CVARC	constant for geometry elements: Arc
L	CVPOLYGON	constant for geometry elements: polygon
L	CVGEOLIST	constant for geometry elements: GeoList
L	CVHERMITEPROBLEM	constant for geometry elements: HermiteProb
L	CVHERMITEPROBLEMSYMMETRIC	Const. for geometry elements: SymHermiteProb
L	CVBIARC	Const. for geometry elements: Biarc
E	GetNullEps	return of the zero comparison command.
E	GetPoint	identifier for points
E	GetLine	Identifier for lines
E	GetArc	identifier for circular arcs
E	GetPolygon	identifier for polygons
E	GetGeoList	Identifier for geometry lists
E	GetHermiteProblemSymmetric	Identifier for symmetric Hermite problems
E	GetHermiteProblem	ID for Hermite problems
E	GetBiarc	identifier for Biarcs

26.5.10. Module MGeneralMath

MGeneralMath is a module for general mathematical functions and works with the data structures and procedures.

MeneralMath

E	MPoint	Data structure for a point
E	MPointX	Reading of the x-coordinate for a point
E	MPointY	reading the y-coordinate for one point
L	MPointIllustrateXY	plot structure for a blue point
E	MPointIllustrate	plot structure for a blue point
E	MPointPlot	Illustration of a green point
E	MLine	Data structure for a line
E	MLineStartPoint	Reading of the start point for a draw frame
E	MLineEndPoint	Reading the end point for a line
E	MPointOnLine	Calculation of a point on the line
E	MLinePlot2D	Plot the part of a line starting at the starting point
. E	MLineLine	calculating the intersection of two linesline

26.5.11. Module Biarc

Data Structure

Requirements and specifications:

The Biarc is to be used to solve a Hermite problem. A Hermite problem is defined as follows:

Given two points P_0 and P_1 with associated normalised tangents $\vec{t_0}$ and $\vec{t_1}$. The biarc must connect the points such that the tangents of the start and end points of the biarc coincide with the tangents of the Hermites problem.

Data structure:

The data structure `New` for a biarc must contain two arcs.

So the data structure for one arc is needed: `MArc:-New`.

This in turn contains the coordinates for the centre, the radius, the start angle and the angle change.

26.5.12. Module MBiarc

`MBiarc` is a module for Biarcs and works with a data structure and with procedures/functions. The data structure `New` for a Biarc is a list which is structured as follows:

```
[MVBIARC, [Arc0, Arc1]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is „Biarc“ and the elements are two arcs.

Using the Get functions `GetArc0` and `GetArc1` one can capture the two arcs for the biarc. The functions listed below can be used to calculate the data for the biarc.

MBiarc		
E	New	Data structure for a biarc
E	GetArc0	Reading of the first arc
E	GetArc1	Reading the second arc
E	Circle	calculating the circle K_J from the Hermite problem data.
E	Plot2DCircle	plot of the circle K_J
E	angle	Calculate the angle from the centre of the circle to points on the circle
E	Plot2D	Plot of the biarc
E	ConnectionPoint	Calculation of the connection point J (Equal Chord)
E	TangentTj	Calculation of the tangent to J
E	Tangent Biarc	rotation of Tj for the biarc.
E	BiarcCenter	Calculate the centres of the arcs of the biarc
E	Biarc	calculate the centre of the biarc.
E	Position	Determine a point on the biarc
E	Blend	Calculate the biarc only from the Hermite problem

26.6. Programme Flowchart

26.6.1. Overall Flow

26.6.2. Verrundung der Kurve

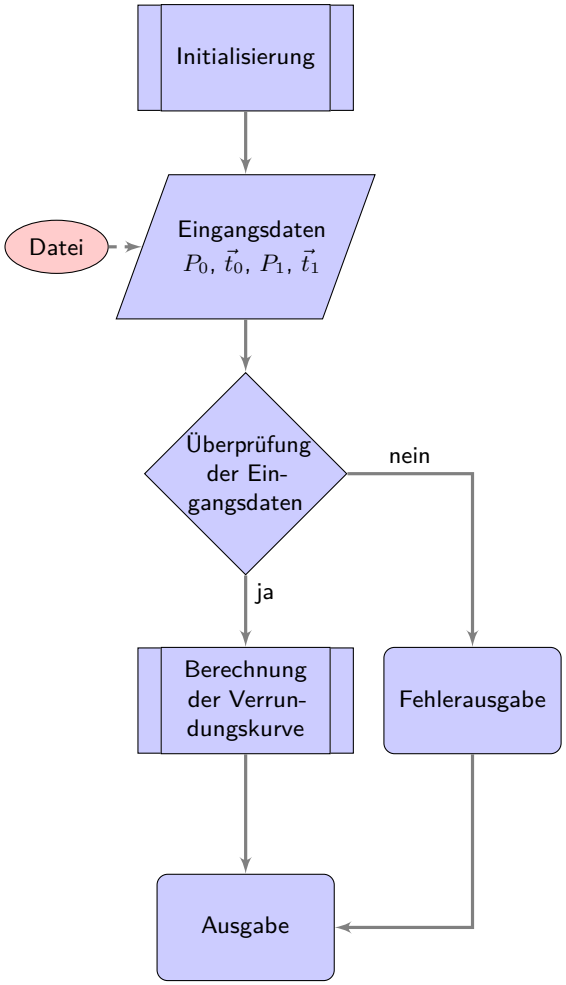


Figure 26.1.: Programme flow chart „Corner rounding“

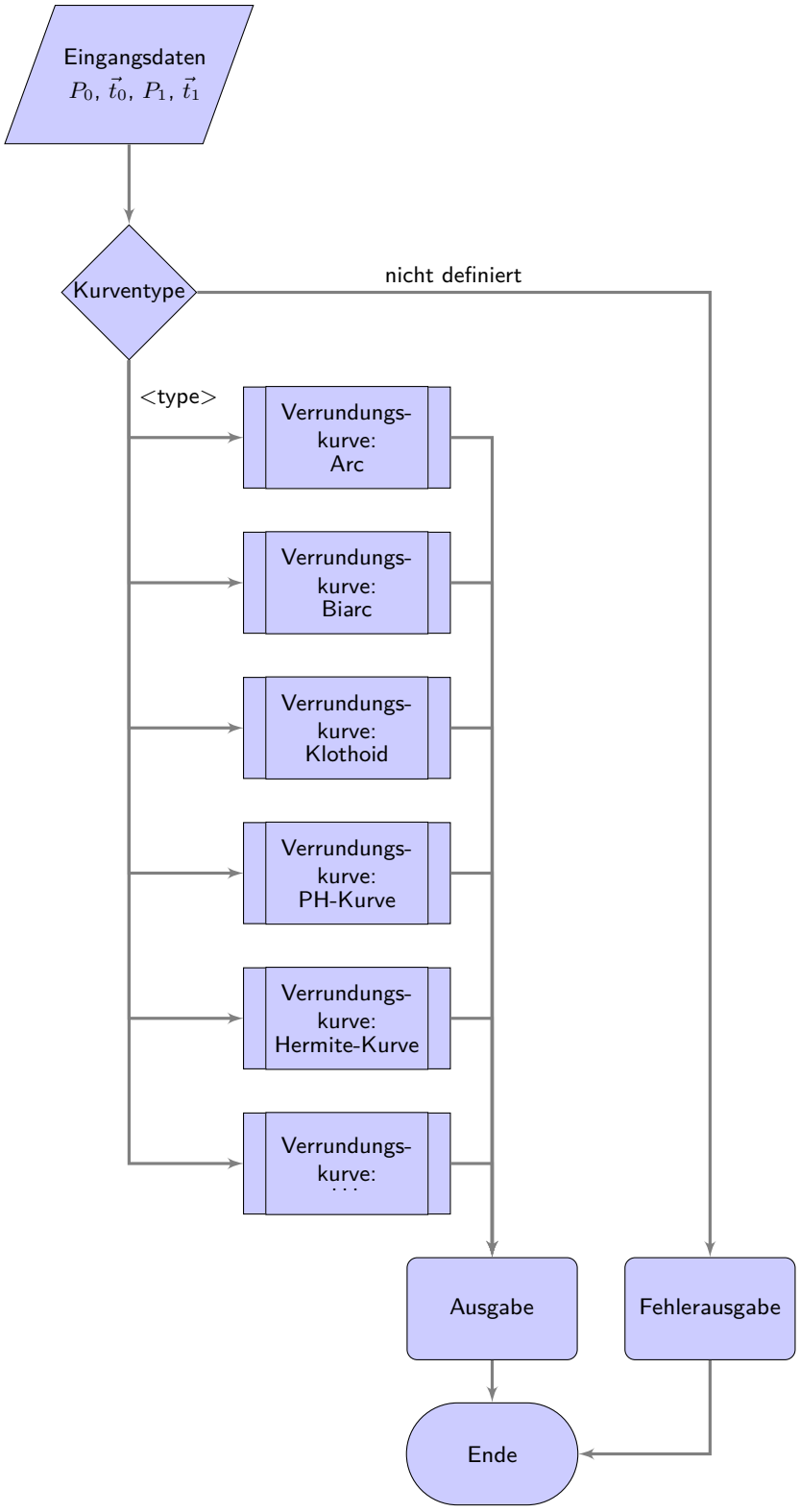


Figure 26.2.: Programme flowchart „Selection of the rounding strategies“

27. Representation of Python Programs

It is possible to integrate a part into the document, see 27.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document. It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 27.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file `Blink.py`.

```
%%
%% This is file `rename-to-empty-base.tex',
%% generated with the docstrip utility.
%%
%% The original source files were:
%%
%% fileerr.dtx  (with options: `return')
%%
%% This is a generated file.
%%
%% The source is maintained by the LaTeX Project team and bug
%% reports for it can be opened at https://latex-project.org/bugs/
%% (but please observe conditions on bug reports sent to that address!)
%%
%%
%% Copyright (C) 1993-2025
%% The LaTeX Project and any individual authors listed elsewhere
%% in this file.
%%
%% This file was generated from file(s) of the Standard LaTeX `Tools
    Bundle'.
%%
%% ----------------------------------------------------------------------
%%
%% It may be distributed and/or modified under the
%% conditions of the LaTeX Project Public License, either version 1.3c
%% of this license or (at your option) any later version.
%% The latest version of this license is in
%%    https://www.latex-project.org/lppl.txt
%% and version 1.3c or later is part of all distributions of LaTeX
%% version 2005/12/01 or later.
%%
%% This file may only be distributed together with a copy of the LaTeX
%% `Tools Bundle'. You may however distribute the LaTeX `Tools Bundle'
%% without such generated files.
%%
%% The list of all files belonging to the LaTeX `Tools Bundle' is
%% given in the file `manifest.txt'.
%%
 \message{File ignored}
\endinput
%%
%% End of file `rename-to-empty-base.tex'.
```

../../Code/HelloWorld/Blink.py

28. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 28.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 28.1.: The program “Hello World” for an Arduino microcontroller boardis inserted from the file Blink.ino.

```
1000 /**
     *
1002 *  @file Blink.ino
     *
1004 *  @brief Simple program for testing the configuration
     *
1006 *
     *  Turns an LED on for one second, then off for one second, repeatedly.
1008 *
     *  Most Arduinos have an on-board LED you can control. On the UNO, MEGA
         and ZERO
1010 *  it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is
         set to
     *  the correct LED pin independent of which board is used.
1012 *  If you want to know what pin the on-board LED is connected to on your
         Arduino
     *  model, check the Technical Specs of your board at:
1014 *
     *  https://www.arduino.cc/en/Main/Products
1016 *
     *  modified 8 May 2014
1018 *  by Scott Fitzgerald
     *  modified 2 Sep 2016
1020 *  by Arturo Guadalupi
     *  modified 8 Sep 2016
1022 *  by Colby Newman
     *
1024 *  This example code is in the public domain.
     *
1026 *  https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
     */
1028


1030


1032 /**
     *  @brief the setup function runs once when you press reset or power the
         board
1034 *
     *  standard function of Arduino sketches
1036 *
     *  Initialization of the pin LED_BUILTIN as output
1038 *
     *  @param ---
1040 *
     *  @return void
1042 */
     void setup() {
1044   // initialize digital pin LED_BUILTIN as an output.
       pinMode(LED_BUILTIN, OUTPUT);
1046 }

1048 /**
     *  @brief the loop function runs over and over again forever
1050 *
     *  standard function of Arduino sketches
1052 *
     *  swichting the led on / off for 1sec.
1054 *
     *  @param ---
1056 *
     *  @return void
1058 */

1060 void loop() {
       digitalWrite(LED_BUILTIN, HIGH);    // turn the LED on (HIGH is the
         voltage level)
1062   delay(1000);                        // wait for a second
       digitalWrite(LED_BUILTIN, LOW);     // turn the LED off by making the
         voltage LOW
1064   delay(1000);                        // wait for a second
     }
```

../../Code/Arduino/Blink/Blink.ino

29. First Chapter

...
A Speicherprogrammierbare Steuerung (SPS) is ...

A Computerized Numerical Control (CNC) needs a SPS (PLC) to ...

30. CAGD

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.
The book CAGD by Gerald Farin is a classic on splines. [Far02]

The standard 66025 for programming CNC machines is also a classic; however, it does not deal with splines. [DIN66025-2]

Mr. F. Farouki has dealt with both CNC machine programming and splines. His article[1] on a real-time interpolator also shows this. [FS17]

A new dimension of machine tools have emerged with the invention of 3D printers. [Rus+07]

Another aspect of automation technology is communication. Another milestone has been reached with 5G technology. another milestone has been reached.[2]

[1] co-author is J. Srinathu
[2] Zaf+20.

A. drawings with tikz

The package tikz is a powerful tool for creating graphics. Many introductions exist. Here only the first steps are shown, so that you can easily create flowcharts.

Drawing a line and arrows

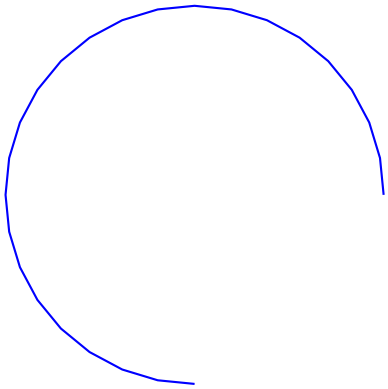
```
\begin{tikzpicture}
    \draw (0,0) -- (1,0);

    \draw[->] (2,0) -- (3,0);

    \draw[<->] (5,0) -- (6,0);
\end{tikzpicture}
```


Drawing a thick blue line

```
\begin{tikzpicture}
    \draw[line width=2pt, blue] (0,0) -- (1,0);

    \draw[line width=2pt, red, dotted] (2,0) -- (3,0);

    \draw[line width=2pt, dashed, green] (4,0) -- (5,0);

    \draw [thick,dash dot] (0,1) -- (5,1);
    \draw [thick,dash pattern={on 7pt off 2pt on 1pt off 3pt}] (0,2) --
        (5,2);
\end{tikzpicture}
```

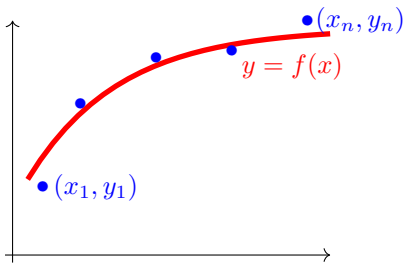
Drawing an arc

```
\begin{tikzpicture}
  \draw [blue,thick,domain=0:270] plot ({5+2.5*cos(\x)}, {1+2.5*sin(\x)});
\end{tikzpicture}
```



Draw a function

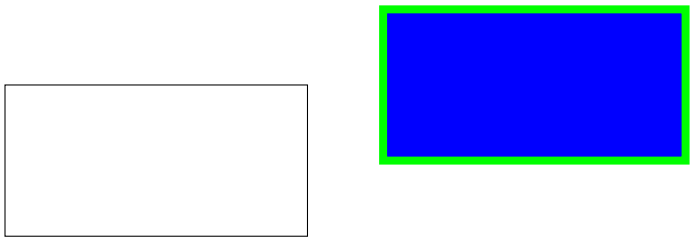
```
\begin{tikzpicture}[
    declare function={%
        F(\x)               =3-2*pow(2.7979,-0.8*\x);
    }
 ]


    % Zeichnen der Funktion
    \draw[red,line width=2pt,domain=0:4] plot({\x},{F(\x)});

    % Bezeichnung
    \node[red] (O) at(3.5,2.5) {$y=f(x)$};

    % Punkte
    \node[blue] (P1n) at (0.9,0.9) {$(x_1,y_1)$};
    \node[blue] (P1) at (0.2,0.9) {$\bullet$};

   \node[blue] (P2) at (2.7,2.7) {$\bullet$};
   \node[blue] (P3) at (1.7,2.6) {$\bullet$};
   \node[blue] (P4) at (0.7,2) {$\bullet$};

   \node[blue] (P5n) at (4.4,3.1) {$(x_n,y_n)$};
   \node[blue] (P5) at (3.7,3.1) {$\bullet$};

   % Koordinatensystem
   \draw[color=black,->] (-0.2,-0.1) -- (-0.2,3.1);
   \draw[color=black,->] (-0.3,0) -- (4,0);
\end{tikzpicture}
```



Drawing rectangles and moving objects

```
\begin{tikzpicture}
    \draw (0,0) -- (4,0)  -- (4,2) -- (0,2) -- cycle;
```

```
    \begin{scope}[shift={(5,1)}]
      \draw[green,fill=blue,line width=3pt] (0,0) -- (4,0)  -- (4,2) --
          (0,2) -- cycle;
    \end{scope}
\end{tikzpicture}
```

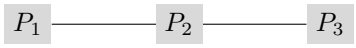


Use of variables

```
\begin{tikzpicture}
\pgfmathsetmacro{\PHI}{-15}
% Now use \PHI anywhere you want -15 to appear,
% can also be used in calculations like 2*\PHI
\def\x{10};

\draw[red] (0,4) -- (1-\PHI*0.5,4);
\draw[green] (0,2) -- (1+1/\x,2);
\draw[blue] (0,0) -- ({1+3*(\x/5+1)},0);
\end{tikzpicture}

\bigskip

%\usetikzlibrary{math} %needed tikz library

\begin{tikzpicture}
  \pgfmathsetmacro{\drehpunktx}{11.63815573}
  %Variables must be declared in a tikzmath environment but
  % can be used outside
  \tikzmath{\x1 = 1; \y1 =1;
  %computations are also possible
  \x2 = \x1 + 1; \y2 =\y1 +3; }
  \draw[->] (\x1, \y1)--(\x2, \y2);
\end{tikzpicture}
```

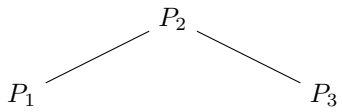
Use of points

```
%\usetikzlibrary{backgrounds}  % is needed

  \begin{tikzpicture}
    \node [fill=gray!30] (P1) at (0,0) { $P_1$ };
    \node [fill=gray!30] (P2) at (2,0) { $P_2$ };
    \node [fill=gray!30] (P3) at (4,0) { $P_3$ };

    \begin{scope}[on background layer]
        \draw (P1) -- (P3);
    \end{scope}
  \end{tikzpicture}
```



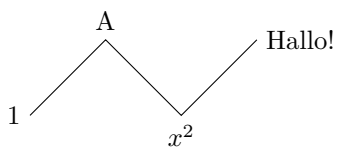
Use of nodes

```
  \begin{tikzpicture}
    \node  (P1) at (0,0){$P_1$};
    \node  (P2) at (2,1){$P_2$};
    \node  (P3) at (4,0){$P_3$};

    \begin{scope}
        \draw (P1) -- (P2) -- (P3);
    \end{scope}
\end{tikzpicture}
```



Use of nodes

```
\begin{tikzpicture}
  \draw (0, 0) node[left]{1}
        -- ++(1, 1) node[above] {A}
        -- ++(1,-1) node[below] {$x^2$}
        -- ++(1, 1) node[right] {Hallo!};
\end{tikzpicture}
```



B. Criteria for a good LaTeX project

A good report is not only characterised by good content, but also fulfils formal aspects. The following list should help to comply with basic rules. Before submitting, all points should be checked and ticked off.

- ☐ Is a suitable directory structure used?
- ☐ Is an appropriate division into files used?
- ☐ Are meaningful directory and file names used?
  - ☐ Are directory and file names such as report or term paper avoided?
  - ☐ Are meaningful names used for images and not „image1“?
  - ☐ Are directory and file names not too long?
  - ☐ Are special characters and spaces avoided?
- ☐ Are commands like \newline and \\ avoided?
- ☐ Are the LaTeXfiles clearly laid out?
  - ☐ Are indentations used in the LaTeXfiles?
  - ☐ Are free lines inserted?
  - ☐ Is the project mentioned in the header of the files?
  - ☐ Does the header of the files mention the main sources?
  - ☐ Is the author mentioned in the header of the files?
- ☐ Are all necessary files given?
- ☐ Are the temporary files deleted?
- ☐ Are graphics created by yourself?
- ☐ Are the sources of the images given?
- ☐ Are citations made with the command \cite?
- ☐ Is a bib file used?
- ☐ Are the entries of the bib-file clearly arranged?
- ☐ Are the entries of the bib-file edited to show them correctly?
- ☐ Are the correct types used for the bib entries?
- ☐ Are meaningful keys used in the bib files?

Unfortunately, the list cannot be complete, but it provides some clues.

C. Model Card

- Model Details - Basic information about the model
  - Person or organization developing model
  - Model date
  - Model version
  - Model type
  - Information about training algorithms, parameters, fairness constraints or other applied approaches, and features
  - Paper or other resource for more information
  - Citation details
  - License
  - Where to send questions or comments about the model
- Intended Use. Use cases that were envisioned during development.
  - Primary intended uses
  - Out-of-scope use cases
- Factors: Factors could include demographic or phenotypic groups, environmental conditions, technical attributes, or others
  - Relevant factors
  - Evaluation factors
- Metrics: Metrics should be chosen to reflect potential real world impacts of the model.
  - Model performance measures
  - Decision thresholds
  - Variation approaches
- Evaluation Data: Details on the dataset(s) used for the quantitative analyses in the card.
  - Datasets
  - Motivation
  - Preprocessing
- Training Data: May not be possible to provide in practice
  - When possible, this section should mirror Evaluation Data. If such detail is not possible, minimal allowable information should be provided here, such as details of the distribution over various factors in the training datasets.
- Quantitative Analyses
  - Unitary results
  - Intersectional results
- Ethical Considerations
- Caveats and Recommendation

Literature

[Aba+16] M. Abadi et al. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

[Ard24] Arduino Documentation. *Getting Started with Arduino IDE 2*. 2024. URL: https://docs.arduino.cc/software/ide-v2/tutorials/getting-started-ide-v2/.

[DIN66025-2] DIN Deutsches Institut für Normung e.V. *DIN 66025-2:1988-09: Programmaufbau für numerisch gesteuerte Arbeitsmaschinen: Wegbedingungen und Zusatzfunktionen*. Norm. Berlin, Sept. 1988.

[FAD18] M. Fezari and A. Al Dahoud. "Integrated Development Environment "IDE" For Arduino". In: (Oct. 2018).

[FS17] R. Farouki and J. Srinathu. "A real-time CNC interpolator algorithm for trimming and filling planar offset curves". In: *Computer-Aided Design* 86 (Jan. 2017). DOI: 10.1016/j.cad.2017.01.001.

[Far02] G. Farin. *Curves and Surfaces for CAGD*. 5. [pdf]. San Diego, CA: Academic Press, 2002.

[Jak+10] G. Jaklic et al. "On Interpolation by Planar Cubic G^2 Pythagorean-Hodograph Spline Curves". In: *Mathematics of Computation* (2010). [pdf].

[Rus+07] D. Russell et al. *Apparatus and Methods for 3D Printing*. United States Patent, Patent N0.: US 7,291,002 B2. 2007.

[Syd] *Hypatia - Handbook*. [pdf]. 2024.

[Sys24] E. Systems, ed. *ESP32S3WROOM1 - ESP32S3WROOM1U*. [pdf]. 2024. URL: https://www.espressif.com/en/company/about-espressif (visited on 11/09/2024).

[Wat17a] Waterloo Maple Inc. 2017. *Description*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/description.

[Wat17b] Waterloo Maple Inc. 2017. *Export*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/export.

[Wat17c] Waterloo Maple Inc. 2017. *Module*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/maple/view.aspx?path=module.

[Wat17d] Waterloo Maple Inc. 2017. *ModuleLoad*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=ModuleLoad.

[Wat17e] Waterloo Maple Inc. 2017. *Option*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/option.

[Wat17f] Waterloo Maple Inc. 2017. *Procedures*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=procedure.

[Zaf+20] A. Zafeiropoulos et al. "Benchmarking and Profiling 5G Verticals' Applications: An Industrial IoT Use Case". In: *IEEE Conference on Network Softwarization, NetSoft 2020*. 2020. DOI: 10.1109/NetSoft48620.2020.9165393.

Index